

Optimization

Adam Kelly (ak2316@cam.ac.uk)

July 28, 2026

Optimization is the art of doing the best you can with what you have got. Almost every quantitative question you might ask about the world – how should a factory allocate its raw materials, how should goods be routed through a road network, how should two adversaries play a game – can be phrased as the problem of making some function as small (or as large) as possible, subject to a list of constraints on what you are allowed to do. The subject is about turning that vague desire into a precise mathematical problem, and then actually *solving* it.

Two themes run through everything we do. The first is *convexity*: for a convex problem, local information is global information, and this is what makes optimization tractable at all. The second is *duality*: every optimization problem casts a shadow, a second problem whose solution certifies the optimality of the first. Duality is what lets us know when we are done, and it appears in different disguises as complementary slackness, as shadow prices, as the max-flow min-cut theorem and as the minimax theorem of game theory. Along the way we will develop honest algorithms – gradient descent, the simplex method, the transportation algorithm, Ford–Fulkerson – that can be run by hand or by machine.

This article constitutes my notes for the ‘Optimization’ course, held in Easter 2021 at Cambridge. These notes are *not a transcription of the lectures*, and differ significantly in quite a few areas. Still, all lectured material should be covered.

Contents

§1 Convex Optimization

§1.1 Optimization Problems

In this course, we care about problems that look like ‘minimize $f(x)$ subject to $x \in X$ ’, or problems with constraints that look like ‘minimize¹ $f(x)$ subject to $x \in X$ where $h(x) = b$ ’.

In such problems, we call f the **objective function**, and if $x \in \mathbb{R}^n$, we call the components **decision variables**. Constraints such as the one above can be given as a **functional constraint** $h(x) = b$ and a **regional constraint** $x \in X$. If both of the constraints hold for a given x , we say it is in the **feasible set** $X(b)$. A problem is feasible if the feasible set is non-empty, and bounded if the minimum over this set is bounded.

A point $x^* \in X(b)$ is **optimal** if it minimizes f over $X(b)$. The value $f(x^*)$ is the **optimal cost**.

¹We will almost always talk about minimizing, since maximizing is basically the same thing: to maximize f we can minimize $-f$.

It is worth setting up some notation for all of this, since we are going to be writing down an awful lot of optimization problems. We will write a problem in the form

$$\begin{aligned} &\text{minimize} && f(x) \\ &\text{subject to} && h(x) = b, \\ &&& x \in X, \end{aligned}$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$, $b \in \mathbb{R}^m$ and $X \subseteq \mathbb{R}^n$. So the feasible set is

$$X(b) = \{x \in X : h(x) = b\}.$$

Remark. The split between ‘functional’ and ‘regional’ constraints is not intrinsic to the problem – it is a choice that we make. The same set of feasible points can very often be described in several ways, and a large part of the skill in this course lies in choosing the description that makes the problem easiest to attack. Typically the regional constraint $x \in X$ will be something simple like $x \geq 0$ or $x \in \mathbb{Z}^n$, and the functional constraints will be the interesting ones that we handle with Lagrange multipliers.

At first glance it looks as though we have restricted ourselves to *equality* constraints, which would be a serious limitation. In fact we have not: an inequality constraint can always be converted into an equality constraint at the cost of introducing a new variable. If we want $h(x) \leq b$, we introduce a **slack variable** s and write

$$h(x) \leq b \quad \longleftrightarrow \quad h(x) + s = b, \quad s \geq 0.$$

The new variable s measures exactly how much ‘slack’ there is in the constraint, and the sign condition $s \geq 0$ is pushed into the regional constraint. This trick will be used constantly.

Example 1.1

Consider the problem of minimizing $x_1^2 + x_2^2$ subject to $x_1 + x_2 \geq 1$. Here $f(x) = x_1^2 + x_2^2$, and we can take $X = \mathbb{R}^2$ with the single functional constraint $x_1 + x_2 \geq 1$. Introducing a slack variable, this becomes: minimize $x_1^2 + x_2^2$ subject to $x_1 + x_2 - s = 1$ and $(x_1, x_2, s) \in \mathbb{R}^2 \times [0, \infty)$. The feasible set is a closed half-plane, and the optimum is at $x = (1/2, 1/2)$ with optimal cost $1/2$ – the point of the half plane closest to the origin.

§1.2 Convex Functions

We will first study optimization problems involving convex functions without constraints. In particular, we are going to study the problem:

$$\text{Minimize } f(x), \text{ where } f : \mathbb{R}^n \rightarrow \mathbb{R} \text{ is a convex function.}$$

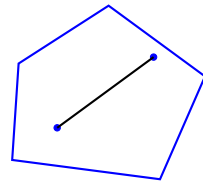
The reason we will care specifically about convex functions is, as you will see, that local information about the function gives us global information about the function.

It is likely that you are already familiar with the notions of convex sets and convex functions.

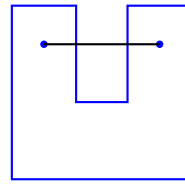
Definition 1.2 (Convex Set)

A set $S \subseteq \mathbb{R}^n$ is **convex** if for all $x, y \in S$ and all $\lambda \in [0, 1]$ we have $\lambda x + (1 - \lambda)y \in S$.

Informally, a set is convex if the line segment between any two points is totally contained in the set.



convex



not convex

We can also define convexity for functions.

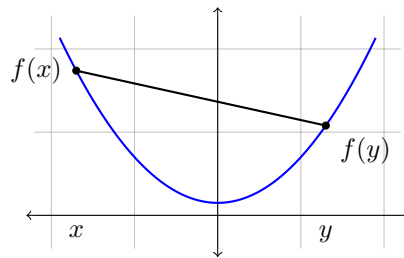
Definition 1.3 (Convex Function)

A function $f : S \rightarrow \mathbb{R}$ is **convex** if S is convex and for all $x, y \in S$ and $\lambda \in [0, 1]$ we have

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

If this inequality is strict (for $x \neq y$ and $\lambda \in (0, 1)$) then the function is **strictly convex**. A function is called **concave** if $-f$ is convex.

Again informally, this says that if we consider the graph of the function then the chord between two points will always lie above the function.



Note that a linear function $f(x) = c \cdot x$ is both convex and concave, since the defining inequality holds with equality. So linear programming, which we will spend a lot of time on later, is a special case of convex optimization.

In general, checking the definition by hand is hard, and we typically use some other conditions to check for convexity.

Proposition 1.4 (First-Order Conditions for Convexity)

A differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if and only if for all $x, y \in \mathbb{R}^n$ we have

$$f(y) \geq f(x) + (y - x) \cdot \nabla f(x).$$

Proof. We will first check that convexity implies that this condition holds by showing it holds for $n = 1$ and using that for the general case.

If $f : \mathbb{R} \rightarrow \mathbb{R}$ is convex, and $x, y \in \mathbb{R}$ then for any $t \in (0, 1]$ we have

$$\begin{aligned} f((1-t)x + ty) &= f(x + t(y-x)) \\ &\leq (1-t)f(x) + tf(y) \\ \implies f(y) &\geq f(x) + \frac{f(x + t(y-x)) - f(x)}{t}. \end{aligned}$$

Taking the limit as $t \rightarrow 0$, the quotient tends to $f'(x)(y - x)$, and so $f(y) \geq f(x) + f'(x)(y - x)$.

Now for $f : \mathbb{R}^n \rightarrow \mathbb{R}$, taking any $x, y \in \mathbb{R}^n$ we can define the restricted function $g(t) = f((1 - t)x + ty)$. Since f is convex, g is also convex. We can then compute

$$g'(t) = (y - x) \cdot \nabla f((1 - t)x + ty).$$

So using the $n = 1$ case, we have $g(1) \geq g(0) + g'(0)(1 - 0)$, that is, $f(y) \geq f(x) + (y - x) \cdot \nabla f(x)$, so the condition holds as required.

Now we will show that the first-order conditions imply convexity. For some $t \in [0, 1]$, we define $x_t = (1 - t)x + ty$. Then we know that

$$\begin{aligned} f(x) &\geq f(x_t) + (x - x_t) \cdot \nabla f(x_t), \\ f(y) &\geq f(x_t) + (y - x_t) \cdot \nabla f(x_t). \end{aligned}$$

Multiplying the first by $(1 - t)$ and the second by t and summing, the gradient terms combine to give

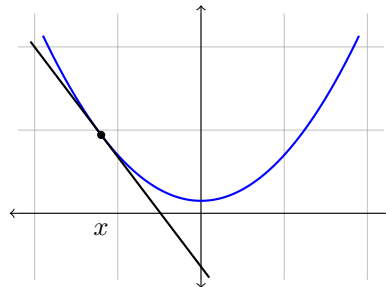
$$(1 - t)(x - x_t) + t(y - x_t) = (1 - t)x + ty - x_t = 0,$$

so we are left with

$$(1 - t)f(x) + tf(y) \geq f(x_t),$$

as required. □

This condition has a very clean geometric meaning: the tangent plane to the graph of f at any point lies entirely *below* the graph.



This picture makes the key fact about convex minimization obvious. If $\nabla f(x) = 0$ at some point x , then the first-order condition reads $f(y) \geq f(x)$ for *all* y , so x is a global minimum. In other words, for a convex function there is no such thing as a bad local minimum: every stationary point is a global minimum. This is exactly the promised phenomenon of local information giving global information, and it is the reason we can hope to minimize convex functions by walking downhill.

A possibly more familiar condition for convexity is the second-order condition. This involves the *Hessian*, the generalization of the second derivative to $\mathbb{R}^n$.

Definition 1.5 (Hessian)

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a twice differentiable function, then the **Hessian matrix** $\nabla^2 f$ is

the symmetric matrix of second derivatives given by

$$(\nabla^2 f)_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}.$$

In particular, we care about a specific property of the Hessian matrix.

Definition 1.6 (Positive Semidefinite)

A symmetric matrix $A \in \mathbb{R}^{n \times n}$ is **positive semidefinite** if for all $x \in \mathbb{R}^n$ we have $x \cdot Ax \geq 0$; equivalently, if all of the eigenvalues of A are non-negative. We write $A \succeq 0$. We write $A \succeq B$ to mean $A - B \succeq 0$.

Proposition 1.7 (Second-Order Conditions for Convexity)

A twice differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is convex if $\nabla^2 f(x)$ is positive semidefinite at all $x \in \mathbb{R}^n$.

Proof. Using the Taylor expansion of f , we have

$$f(y) = f(x) + (y - x) \cdot \nabla f(x) + \frac{1}{2}(y - x) \cdot (\nabla^2 f(z)(y - x)),$$

where $z = (1 - \lambda)x + \lambda y$ for some $\lambda \in [0, 1]$. Since $\nabla^2 f(z) \succeq 0$, the final term is non-negative, and so

$$f(y) \geq f(x) + (y - x) \cdot \nabla f(x).$$

Then by the first-order conditions, f is convex. $\square$

The converse also holds, but we will not need it.

Example 1.8

Let us see the conditions in action.

- (i) $f(x) = x \cdot Ax$ for a symmetric matrix A has $\nabla^2 f = 2A$, so f is convex exactly when $A \succeq 0$.
- (ii) $f(x_1, x_2) = x_1^2 + 100x_2^2$ has Hessian $\text{diag}(2, 200) \succeq 0$, so is convex.
- (iii) $f(x) = e^{c \cdot x}$ has $\nabla^2 f = (cc^T)e^{c \cdot x}$, and $y \cdot (cc^T)y = (c \cdot y)^2 \geq 0$, so f is convex for any c .
- (iv) $f(x_1, x_2) = x_1x_2$ has Hessian $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, whose eigenvalues are ± 1 . So this function is neither convex nor concave.

§1.3 Gradient Descent

To solve our problem of minimizing $f(x)$ where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is a convex function, we can begin by noting that convexity implies that a local minimum is a global minimum. So if we find a point where the gradient of the function is 0, then we would be finished. This suggests an extremely simple ‘greedy’ strategy: start somewhere, look around for a nearby point with a smaller value of f , move there, and repeat.

To make this concrete we need to know which direction to move in. Taking a small

step of size ε in the direction v and Taylor expanding,

$$f(x + \varepsilon v) \approx f(x) + \varepsilon v \cdot \nabla f(x),$$

so any v with $v \cdot \nabla f(x) < 0$ will decrease f , at least for small enough ε . We call such a v a **descent direction**. The most natural choice is $v = -\nabla f(x)$, which gives

$$f(x - \varepsilon \nabla f(x)) \approx f(x) - \varepsilon \|\nabla f(x)\|^2 \leq f(x).$$

Definition 1.9 (Gradient Descent)

The **gradient descent** algorithm for minimizing f is as follows.

- (1) Choose a starting point x_0 and set $t = 0$.
- (2) If $\nabla f(x_t) = 0$ (or we have run out of patience), stop and return x_t .
- (3) Choose a step size $\eta_t > 0$ and set $x_{t+1} = x_t - \eta_t \nabla f(x_t)$.
- (4) Increment t and go to (2).

The whole art is in the choice of η_t . If it is too small we crawl; if it is too large we overshoot and oscillate. To say anything precise we need to control how fast the gradient can change.

Definition 1.10 (Smoothness)

A continuously differentiable $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **β -smooth** if ∇f is β -Lipschitz, that is

$$\|\nabla f(x) - \nabla f(y)\| \leq \beta \|x - y\|$$

for all x, y . If f is twice differentiable this is equivalent to $\nabla^2 f(x) \preceq \beta I$ for all x .

Smoothness says that the linear approximation to f cannot be too badly wrong.

Lemma 1.11 (Quadratic Upper Bound)

If f is β -smooth then for all x, y ,

$$f(y) \leq f(x) + (y - x) \cdot \nabla f(x) + \frac{\beta}{2} \|y - x\|^2.$$

Proof. By Taylor's theorem there is a point z on the segment from x to y with

$$f(y) = f(x) + (y - x) \cdot \nabla f(x) + \frac{1}{2} (y - x) \cdot (\nabla^2 f(z)(y - x)),$$

and since $\nabla^2 f(z) \preceq \beta I$ we have $(y - x) \cdot (\nabla^2 f(z)(y - x)) \leq \beta \|y - x\|^2$. $\square$

Together with convexity this sandwiches f between two functions that agree to first order at x : a linear lower bound and a quadratic upper bound. The quadratic upper bound is the useful one, because we can minimize it exactly.

Corollary 1.12

If f is β -smooth and we take a step of size $1/\beta$, then

$$f\left(x - \frac{1}{\beta}\nabla f(x)\right) \leq f(x) - \frac{1}{2\beta}\|\nabla f(x)\|^2.$$

Proof. Minimizing the right-hand side of the quadratic upper bound over y , we differentiate to get $\nabla f(x) + \beta(y - x) = 0$, so the minimizer is $y = x - \frac{1}{\beta}\nabla f(x)$. Substituting this in,

$$\begin{aligned} f(y) &\leq f(x) - \frac{1}{\beta}\|\nabla f(x)\|^2 + \frac{\beta}{2} \cdot \frac{1}{\beta^2}\|\nabla f(x)\|^2 \\ &= f(x) - \frac{1}{2\beta}\|\nabla f(x)\|^2. \square \end{aligned}$$

So each step makes definite progress, provided the gradient is not small. But a small gradient does not by itself mean we are near the optimum – think of a function with a long, almost flat valley. To rule this out we need a lower bound on the curvature as well.

Definition 1.13 (Strong Convexity)

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **α -strongly convex** if for all x, y ,

$$f(y) \geq f(x) + (y - x) \cdot \nabla f(x) + \frac{\alpha}{2}\|y - x\|^2.$$

For twice differentiable f this is equivalent to $\nabla^2 f(x) \succeq \alpha I$ for all x .

Lemma 1.14

Let f be α -strongly convex with minimum value p^* . Then for any x ,

$$p^* \geq f(x) - \frac{1}{2\alpha}\|\nabla f(x)\|^2.$$

Proof. Take the minimum over y of both sides of the strong convexity inequality. The left-hand side becomes p^* . The right-hand side is a quadratic in y , minimized where $\nabla f(x) + \alpha(y - x) = 0$, that is at $y = x - \frac{1}{\alpha}\nabla f(x)$, giving

$$p^* \geq f(x) - \frac{1}{\alpha}\|\nabla f(x)\|^2 + \frac{1}{2\alpha}\|\nabla f(x)\|^2 = f(x) - \frac{1}{2\alpha}\|\nabla f(x)\|^2. \quad \square$$

In other words, if $\|\nabla f(x)\| \leq \sqrt{2\alpha\epsilon}$ then $f(x) \leq p^* + \epsilon$: for a strongly convex function, a small gradient really does mean we are nearly done. Putting the two bounds together gives a genuine convergence guarantee.

Theorem 1.15 (Convergence of Gradient Descent)

Let f be β -smooth and α -strongly convex with $0 < \alpha \leq \beta$, and let x^* be its minimizer. Then gradient descent with step size $1/\beta$ satisfies

$$f(x_T) - f(x^*) \leq \left(1 - \frac{\alpha}{\beta}\right)^T (f(x_0) - f(x^*)) \leq e^{-\alpha T/\beta} \cdot \frac{\beta}{2}\|x_0 - x^*\|^2.$$

Proof. Write $\delta_t = f(x_t) - f(x^*)$. By the corollary above and then the lemma above (applied at x_t , with $p^* = f(x^*)$),

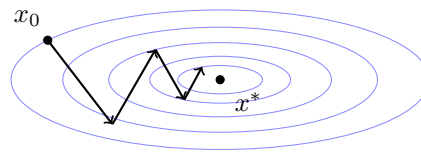
$$\delta_{t+1} \leq \delta_t - \frac{1}{2\beta} \|\nabla f(x_t)\|^2 \leq \delta_t - \frac{\alpha}{\beta} \delta_t = \left(1 - \frac{\alpha}{\beta}\right) \delta_t,$$

where the middle step used $\|\nabla f(x_t)\|^2 \geq 2\alpha\delta_t$. Induction gives the first inequality, and $1 - u \leq e^{-u}$ gives the second. Finally, β -smoothness applied at x^* (where $\nabla f(x^*) = 0$) gives $\delta_0 \leq \frac{\beta}{2} \|x_0 - x^*\|^2$. $\square$

The quantity $\kappa = \beta/\alpha \geq 1$ is called the **condition number** of f . To get $f(x_T) - f(x^*) \leq \varepsilon$ we need about $\kappa \log(1/\varepsilon)$ steps; each extra decimal place of accuracy costs a constant number of extra steps, which is called *linear convergence* and is very fast indeed. But the constant is κ , and if κ is huge then convergence is slow.

Example 1.16

Consider $f(x_1, x_2) = \frac{1}{2}(x_1^2 + 100x_2^2)$, whose Hessian is $\text{diag}(1, 100)$, so $\alpha = 1$, $\beta = 100$ and $\kappa = 100$. The level sets are extremely elongated ellipses, and the gradient at a typical point points almost straight at the long axis rather than at the origin. Gradient descent therefore zig-zags across the valley making very slow progress along it.



§1.4 Newton's Method and Barrier Methods

The problem in the example above is that gradient descent treats all coordinates the same way, whereas the function does not. The fix is to rescale by the Hessian. Recall that the second-order approximation to f near x_t is

$$f(x) \approx f(x_t) + (x - x_t) \cdot \nabla f(x_t) + \frac{1}{2}(x - x_t) \cdot (\nabla^2 f(x_t)(x - x_t)),$$

and minimizing the right-hand side over x gives $x = x_t - (\nabla^2 f(x_t))^{-1} \nabla f(x_t)$.

Definition 1.17 (Newton's Method)

Newton's method is the iteration

$$x_{t+1} = x_t - (\nabla^2 f(x_t))^{-1} \nabla f(x_t).$$

So instead of approximating f by a linear function and stepping a fixed distance, we approximate it by a paraboloid and jump straight to the bottom of that paraboloid. Near the optimum this converges quadratically: one can show that

$$\|x_{t+1} - x^*\| \leq c \|x_t - x^*\|^2$$

once x_t is close enough to x^* , so the number of correct digits roughly doubles each step. The price is that we have to compute and invert an $n \times n$ matrix at every step.

Remark. In one dimension, Newton's method reads $x_{t+1} = x_t - f'(x_t)/f''(x_t)$, which is exactly the familiar Newton–Raphson iteration for finding a root of $g = f'$. Minimizing f and finding a zero of its derivative are of course the same problem.

Finally, a word about constraints. Suppose we want to minimize $f(x)$ subject to $f_i(x) \leq 0$ for $i = 1, \dots, m$. We would love to reduce this to an unconstrained problem, and the obvious way is to minimize

$$f(x) + \sum_{i=1}^m \phi(f_i(x)),$$

where $\phi(y) = 0$ for $y \leq 0$ and $\phi(y) = +\infty$ for $y > 0$. This is exactly the original problem, but ϕ is not remotely differentiable and so all of our machinery fails. Instead we use a smooth approximation.

Definition 1.18 (Logarithmic Barrier)

The **logarithmic barrier method** solves the unconstrained problem

$$\text{minimize} \quad tf(x) - \sum_{i=1}^m \log(-f_i(x))$$

for a sequence of increasing values of $t > 0$, using the previous solution as the starting point for the next.

The function $-\log(-y)$ is $+\infty$ for $y \geq 0$ and finite for $y < 0$, rising steeply as $y \rightarrow 0^-$; it fences the iterates inside the feasible region. As $t \rightarrow \infty$ the objective f dominates the barrier, and the minimizer converges to the solution of the constrained problem. Each subproblem is solved by Newton's method. This is the basic idea behind *interior point methods*, which are what modern solvers actually use.

§2 Lagrangian Methods

Everything in the previous section was about *unconstrained* problems. Real problems come with constraints, and the central idea for handling them – one of the great ideas in the subject – is to not handle them at all. Instead we build the constraints into the objective function with a price attached, and then solve an unconstrained problem. If we get the prices right, the solution of the unconstrained problem automatically satisfies the constraints.

§2.1 The Lagrangian

Throughout this section we consider the problem

$$\begin{aligned} &\text{minimize} && f(x) \\ &\text{subject to} && h(x) = b, \\ &&& x \in X, \end{aligned}$$

where $h : \mathbb{R}^n \rightarrow \mathbb{R}^m$. We will refer to this as the **primal problem**, denoted (P) .

Definition 2.1 (Lagrangian)

The **Lagrangian** of the problem (P) is the function

$$L(x, \lambda) = f(x) - \lambda \cdot (h(x) - b),$$

where $\lambda \in \mathbb{R}^m$ is the vector of **Lagrange multipliers**.

The point is this. If x satisfies the functional constraint $h(x) = b$, then the second term vanishes and $L(x, \lambda) = f(x)$, whatever λ is. So on the feasible set the Lagrangian *is* the objective function. Off the feasible set, the term $-\lambda \cdot (h(x) - b)$ acts as a penalty (or a reward, depending on signs) for violating the constraints, and if we choose λ well then the penalty is enough to keep the unconstrained minimizer of L inside the feasible set. Making this precise gives the following theorem, which will be our main tool.

Theorem 2.2 (Lagrangian Sufficiency)

Suppose we can find $\lambda^* \in \mathbb{R}^m$ and $x^* \in X$ such that

- (i) x^* minimizes $L(\cdot, \lambda^*)$ over all of X , that is, $L(x^*, \lambda^*) = \min_{x \in X} L(x, \lambda^*)$; and
- (ii) x^* is feasible, that is, $x^* \in X(b)$.

Then x^* is optimal for (P) .

Proof. Since x^* is feasible we certainly have $f(x^*) \geq \min_{x \in X(b)} f(x)$, so we only need the reverse inequality. For this, note that if $x \in X(b)$ then $h(x) - b = 0$, so subtracting $\lambda^* \cdot (h(x) - b)$ changes nothing:

$$\begin{aligned} \min_{x \in X(b)} f(x) &= \min_{x \in X(b)} (f(x) - \lambda^* \cdot (h(x) - b)) \\ &\geq \min_{x \in X} (f(x) - \lambda^* \cdot (h(x) - b)) \\ &= \min_{x \in X} L(x, \lambda^*) \\ &= L(x^*, \lambda^*) \\ &= f(x^*) - \lambda^* \cdot (h(x^*) - b) = f(x^*). \end{aligned}$$

Here the inequality is because we are minimizing over a larger set $X \supseteq X(b)$, the next-to-last line is hypothesis (i), and the last line is hypothesis (ii). So $f(x^*) = \min_{x \in X(b)} f(x)$ as claimed. $\square$

This gives us a recipe. Rather than minimize over the awkward set $X(b)$, we minimize over the nice set X for each fixed λ , obtaining a candidate $x^*(\lambda)$, and then hunt for the value of λ making $x^*(\lambda)$ feasible. Notice that we have to be careful: for some λ the Lagrangian will not have a finite minimum at all. So we define

$$\Lambda = \left\{ \lambda \in \mathbb{R}^m : \inf_{x \in X} L(x, \lambda) > -\infty \right\},$$

and only consider $\lambda \in \Lambda$.

Example 2.3

Consider the problem

$$\begin{aligned} & \text{minimize} && -x_1 - x_2 + x_3 \\ & \text{subject to} && x_1^2 + x_2^2 = 4, \\ & && x_1 + x_2 + x_3 = 1, \end{aligned}$$

over $x \in \mathbb{R}^3$. So $X = \mathbb{R}^3$ and $b = (4, 1)$. The Lagrangian is

$$\begin{aligned} L(x, \lambda) &= -x_1 - x_2 + x_3 - \lambda_1(x_1^2 + x_2^2 - 4) - \lambda_2(x_1 + x_2 + x_3 - 1) \\ &= \left(-(1 + \lambda_2)x_1 - \lambda_1 x_1^2\right) + \left(-(1 + \lambda_2)x_2 - \lambda_1 x_2^2\right) + (1 - \lambda_2)x_3 + 4\lambda_1 + \lambda_2. \end{aligned}$$

We have arranged the terms so that each variable appears in exactly one bracket, and so we can minimize each bracket separately.

First, which λ lie in Λ ? The x_3 term is linear, so unless $\lambda_2 = 1$ we can send it to $-\infty$. The x_1 term is a quadratic with leading coefficient $-\lambda_1$, so we need $\lambda_1 \leq 0$; and if $\lambda_1 = 0$ then the remaining linear term $-(1 + \lambda_2)x_1 = -2x_1$ is unbounded below. So $\Lambda = \{\lambda : \lambda_1 < 0, \lambda_2 = 1\}$.

For such λ , minimizing each bracket by differentiating,

$$-(1 + \lambda_2) - 2\lambda_1 x_1 = 0 \implies x_1 = \frac{-2}{2\lambda_1} = \frac{-1}{\lambda_1},$$

and identically $x_2 = -1/\lambda_1$. Note x_3 is completely free (its coefficient is zero), which is exactly what we want – it leaves us room to satisfy the second constraint.

Now we impose feasibility. From $x_1^2 + x_2^2 = 4$ we get $2/\lambda_1^2 = 2$, so $\lambda_1 = -1/\sqrt{2}$ (taking the negative root as required) and $x_1 = x_2 = \sqrt{2}$. Then the second constraint gives $x_3 = 1 - 2\sqrt{2}$.

By Lagrangian sufficiency, $x^* = (\sqrt{2}, \sqrt{2}, 1 - 2\sqrt{2})$ is optimal, with optimal cost $-1 - 2\sqrt{2}$.

§2.2 Inequality Constraints and Complementary Slackness

Now suppose the constraints are inequalities:

$$\begin{aligned} & \text{minimize} && f(x) \\ & \text{subject to} && h(x) \leq b, \\ & && x \in X. \end{aligned}$$

We already know how to deal with this – add a slack variable $s \geq 0$ and write $h(x) + s = b$. The Lagrangian becomes

$$L(x, s, \lambda) = f(x) - \lambda \cdot (h(x) + s - b),$$

and we now minimize over $x \in X$ and $s \geq 0$. Writing the Lagrangian as

$$L(x, s, \lambda) = \underbrace{f(x) - \lambda \cdot (h(x) - b)}_{\text{does not involve } s} - \lambda \cdot s,$$

we can read off two things immediately.

Firstly, since s ranges over all of $[0, \infty)^m$, the term $-\lambda \cdot s$ is bounded below only if $\lambda \leq 0$. So $\lambda \in \Lambda$ forces $\lambda \leq 0$: with the sign convention above, *Lagrange multipliers for $\leq$ constraints are non-positive*.

Secondly, given $\lambda \leq 0$, the minimum over $s \geq 0$ of $-\lambda \cdot s$ is attained by taking $s_i = 0$ whenever $\lambda_i < 0$ (and s_i arbitrary when $\lambda_i = 0$). In either case the minimizing s satisfies $\lambda_i s_i = 0$ for every i . This is important enough to have a name.

Theorem 2.4 (Complementary Slackness)

At an optimum obtained by the Lagrangian method, for each i we have

$$\lambda_i s_i = 0, \quad \text{equivalently} \quad \lambda_i (h_i(x) - b_i) = 0.$$

So for each i , either the constraint is **tight** ($h_i(x) = b_i$) or the multiplier vanishes ($\lambda_i = 0$) – or both.

In words: a constraint that is not binding has zero multiplier. If we are not up against a constraint, then it is not costing us anything, so its price should be zero. We will see this interpretation made precise in a moment, and we will meet the same phenomenon again and again – in linear programming, in the transportation algorithm, and in game theory.

Complementary slackness is enormously useful in practice, because it lets us split into cases: for each constraint, either $s_i = 0$ or $\lambda_i = 0$, and often most of the cases can be eliminated quickly.

Example 2.5

Consider

$$\begin{aligned} &\text{minimize} && x_1 - 3x_2 \\ &\text{subject to} && x_1^2 + x_2^2 \leq 4, \\ &&& x_1 + x_2 \leq 2, \end{aligned}$$

over $x \in \mathbb{R}^2$. Adding slack variables $s_1, s_2 \geq 0$, the Lagrangian is

$$\begin{aligned} L &= (x_1 - 3x_2) - \lambda_1(x_1^2 + x_2^2 + s_1 - 4) - \lambda_2(x_1 + x_2 + s_2 - 2) \\ &= ((1 - \lambda_2)x_1 - \lambda_1 x_1^2) + ((-3 - \lambda_2)x_2 - \lambda_1 x_2^2) - \lambda_1 s_1 - \lambda_2 s_2 + 4\lambda_1 + 2\lambda_2. \end{aligned}$$

As discussed, $\lambda_1, \lambda_2 \leq 0$. Minimizing the two brackets,

$$(1 - \lambda_2) - 2\lambda_1 x_1 = 0, \quad (-3 - \lambda_2) - 2\lambda_1 x_2 = 0.$$

If $\lambda_1 = 0$ these read $\lambda_2 = 1$ and $\lambda_2 = -3$, which is a contradiction. So $\lambda_1 < 0$, and complementary slackness gives $s_1 = 0$, i.e. the first constraint is tight: $x_1^2 + x_2^2 = 4$.

Now suppose $\lambda_2 < 0$. Then $s_2 = 0$ too, so $x_1 + x_2 = 2$ as well. Combined with $x_1^2 + x_2^2 = 4$ this gives $(x_1, x_2) = (0, 2)$ or $(2, 0)$. Substituting into the two stationarity equations, the first case gives $(\lambda_1, \lambda_2) = (1, -3)$ and the second gives $(\lambda_1, \lambda_2) = (-1, 1)$; both have a positive multiplier, so both are impossible.

Hence $\lambda_2 = 0$, and the stationarity equations become $x_1 = 1/(2\lambda_1)$ and $x_2 = -3/(2\lambda_1)$. Substituting into $x_1^2 + x_2^2 = 4$,

$$\frac{1 + 9}{4\lambda_1^2} = 4 \implies \lambda_1^2 = \frac{5}{8} \implies \lambda_1 = -\sqrt{5/8},$$

giving

$$(x_1, x_2) = \left(-\sqrt{2/5}, 3\sqrt{2/5} \right).$$

Finally we check that this is feasible for the second constraint: $x_1 + x_2 = 2\sqrt{2/5} \approx 1.26 \leq 2$, so $s_2 = 2 - 2\sqrt{2/5} \geq 0$. By Lagrangian sufficiency this is optimal, with cost $-10\sqrt{2/5} = -2\sqrt{10}$.

Let us record the general procedure, since it is what we will do every time.

Remark (The Lagrangian Recipe). To solve ‘minimize $f(x)$ subject to $h(x) \leq b, x \in X$ ’:

- (1) Introduce slack variables to get equality constraints $h(x) + s = b, s \geq 0$.
- (2) Form the Lagrangian $L(x, s, \lambda) = f(x) - \lambda \cdot (h(x) + s - b)$.
- (3) Find Λ , the set of λ for which $\inf_{x \in X, s \geq 0} L$ is finite.
- (4) For each $\lambda \in \Lambda$, find the minimizers $x^*(\lambda), s^*(\lambda)$.
- (5) Find $\lambda^* \in \Lambda$ for which $x^*(\lambda^*), s^*(\lambda^*)$ is feasible. Then $x^*(\lambda^*)$ is optimal.

§2.3 Lagrangian Duality

The Lagrangian recipe raises an obvious worry: what if there is no λ^* that makes the minimizer feasible? Then the method fails, and we would like to understand when and why. The answer is the theory of duality, which is much more than a diagnostic tool – it is arguably the most important structure in the whole subject.

Definition 2.6 (Dual Function and Dual Problem)

The **dual function** associated with (P) is

$$g(\lambda) = \inf_{x \in X} L(x, \lambda),$$

defined for $\lambda \in \Lambda$. The **dual problem** (D) is

$$\begin{array}{ll} \text{maximize} & g(\lambda) \\ \text{subject to} & \lambda \in \Lambda. \end{array}$$

Note that g is an infimum of *affine* functions of λ (one for each x), and an infimum of affine functions is always concave. So the dual problem is always the maximization of a concave function over a convex set, i.e. always a convex problem, no matter how horrible the primal problem was. This is remarkable and is one reason duality is so useful.

Theorem 2.7 (Weak Duality)

If $x \in X(b)$ and $\lambda \in \Lambda$, then $f(x) \geq g(\lambda)$. In particular

$$\inf_{x \in X(b)} f(x) \geq \sup_{\lambda \in \Lambda} g(\lambda).$$

Proof. This is the first two lines of the proof of Lagrangian sufficiency. For $x \in X(b)$

we have $h(x) = b$, so

$$f(x) = f(x) - \lambda \cdot (h(x) - b) = L(x, \lambda) \geq \inf_{x' \in X} L(x', \lambda) = g(\lambda). \quad \square$$

So every feasible λ gives a *lower bound* on the optimal cost, and every feasible x gives an *upper bound*. This is already valuable: if we find x and λ with $f(x) = g(\lambda)$, we have proved that both are optimal without any further work. In general, though, there may be a gap.

Definition 2.8 (Duality Gap)

The **duality gap** is

$$\inf_{x \in X(b)} f(x) - \sup_{\lambda \in \Lambda} g(\lambda) \geq 0.$$

If the duality gap is zero we say that **strong duality** holds.

The relationship with the Lagrangian method is exactly what one would hope.

Proposition 2.9

The Lagrangian method succeeds – that is, there is a λ^* whose associated minimizer is feasible – if and only if strong duality holds.

Proof. If the method succeeds with multiplier λ^* and minimizer x^* , then by the proof of Lagrangian sufficiency, $g(\lambda^*) = L(x^*, \lambda^*) = f(x^*) = \min_{x \in X(b)} f(x)$, so the gap is zero.

Conversely if the gap is zero then there is λ^* (or a sequence approaching it) with $g(\lambda^*) = \inf_{x \in X(b)} f(x)$, and the chain of inequalities in the sufficiency proof collapses to equalities, so the minimizer of $L(\cdot, \lambda^*)$ is feasible and optimal. $\square$

So we should ask: when does strong duality hold? The answer is beautifully geometric.

§2.4 Supporting Hyperplanes

The key object is the following.

Definition 2.10 (Value Function)

The **value function** of the problem (P) is $\phi : \mathbb{R}^m \rightarrow \mathbb{R} \cup \{\pm\infty\}$ given by

$$\phi(c) = \inf_{x \in X(c)} f(x),$$

the optimal cost when the right-hand side of the functional constraint is changed from b to c .

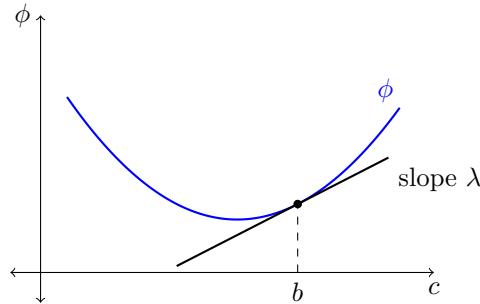
So $\phi(b)$ is the number we are trying to compute, and ϕ tells us how that number responds to perturbing the constraints. Now here is the definition that makes everything work.

Definition 2.11 (Supporting Hyperplane)

A function $\phi : \mathbb{R}^m \rightarrow \mathbb{R}$ has a **supporting hyperplane** at b if there is some $\lambda \in \mathbb{R}^m$ such that

$$\phi(c) \geq \phi(b) + \lambda \cdot (c - b) \quad \text{for all } c \in \mathbb{R}^m.$$

That is, there is a hyperplane through the point $(b, \phi(b))$ which lies entirely below the graph of ϕ . Compare this with the first-order condition for convexity, where the tangent plane at every point lies below the graph. Indeed one can show (we will not prove it) that ϕ is convex if and only if ϕ has a supporting hyperplane at every point.



Theorem 2.12 (Strong Duality)

Strong duality holds for (P) if and only if the value function ϕ has a supporting hyperplane at b .

Proof. Suppose first that ϕ has a supporting hyperplane at b , with normal given by λ . The trick is to compute $g(\lambda)$ by splitting the minimization over $x \in X$ into a minimization over the possible values $c = h(x)$, and then over the x achieving each value:

$$\begin{aligned}
 g(\lambda) &= \inf_{x \in X} (f(x) - \lambda \cdot (h(x) - b)) \\
 &= \inf_{c \in \mathbb{R}^m} \inf_{x \in X(c)} (f(x) - \lambda \cdot (h(x) - c) - \lambda \cdot (c - b)) \\
 &= \inf_{c \in \mathbb{R}^m} \left(\inf_{x \in X(c)} f(x) - \lambda \cdot (c - b) \right) \\
 &= \inf_{c \in \mathbb{R}^m} (\phi(c) - \lambda \cdot (c - b)) \\
 &\geq \phi(b),
 \end{aligned}$$

where in the third line we used that $h(x) = c$ for $x \in X(c)$, and the final inequality is precisely the supporting hyperplane property. But weak duality says $g(\lambda) \leq \phi(b)$, so $g(\lambda) = \phi(b)$ and the duality gap is zero.

Conversely, suppose strong duality holds, so there is λ with $g(\lambda) = \phi(b)$. Then for any c ,

$$\begin{aligned}
 \phi(b) = g(\lambda) &= \inf_{x \in X} (f(x) - \lambda \cdot (h(x) - c) - \lambda \cdot (c - b)) \\
 &\leq \inf_{x \in X(c)} (f(x) - \lambda \cdot (h(x) - c)) - \lambda \cdot (c - b) \\
 &= \phi(c) - \lambda \cdot (c - b),
 \end{aligned}$$

which rearranges to the supporting hyperplane condition. $\square$

This is a clean answer, but it is stated in terms of ϕ , which is the thing we do not know. Fortunately there is a checkable sufficient condition.

Theorem 2.13

Consider the problem ‘minimize $f(x)$ subject to $h(x) \leq b$, $x \in X$ ’, and suppose that X is convex, f is convex and h is convex (componentwise). Then the value function ϕ is convex, and hence strong duality holds.

Proof. Let $b_1, b_2 \in \mathbb{R}^m$ and $\delta \in [0, 1]$, and set $b_\delta = \delta b_1 + (1 - \delta)b_2$. Take any feasible $x_1 \in X(b_1)$ and $x_2 \in X(b_2)$, and let $x_\delta = \delta x_1 + (1 - \delta)x_2$, which lies in X by convexity of X . By convexity of h ,

$$h(x_\delta) \leq \delta h(x_1) + (1 - \delta)h(x_2) \leq \delta b_1 + (1 - \delta)b_2 = b_\delta,$$

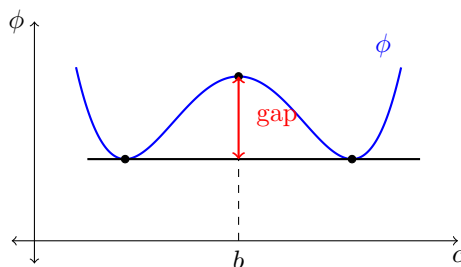
so $x_\delta \in X(b_\delta)$. Hence by convexity of f ,

$$\phi(b_\delta) \leq f(x_\delta) \leq \delta f(x_1) + (1 - \delta)f(x_2).$$

Taking the infimum over $x_1 \in X(b_1)$ and $x_2 \in X(b_2)$ gives $\phi(b_\delta) \leq \delta\phi(b_1) + (1 - \delta)\phi(b_2)$, so ϕ is convex. $\square$

Remark. It is worth stressing where convexity was used. We needed $h(x) \leq b$ rather than $h(x) = b$: with an equality constraint, $h(x_\delta) \leq b_\delta$ does not tell us $h(x_\delta) = b_\delta$, and the argument breaks. This is why in practice one always tries to write constraints as inequalities when hoping for strong duality.

Here, incidentally, is what failure looks like: if ϕ is not convex, there may be a ‘dent’ at b and no hyperplane through $(b, \phi(b))$ can stay below the graph. The best any hyperplane can do is to support the convex hull of the graph from below, and the vertical distance between $\phi(b)$ and that best hyperplane is exactly the duality gap.

**§2.5 Shadow Prices**

We finish this section with the interpretation of Lagrange multipliers that gives them their name in economics, and which explains the strange sign conventions.

Imagine a factory that makes n products out of m raw materials. If it makes quantities $x = (x_1, \dots, x_n)$ then it makes profit $f(x)$ (we will minimize $-f$, but let us think in terms of maximizing profit), and producing x consumes $h_j(x)$ units of raw material j . The factory has b_j units of material j available. So the factory owner solves ‘maximize $f(x)$ subject to $h(x) \leq b$ ’.

Now a supplier turns up and offers a small extra quantity $\varepsilon = (\varepsilon_1, \dots, \varepsilon_m)$ of raw materials. How much should the owner be willing to pay? The new optimal profit is $\phi(b + \varepsilon)$, so the increase is

$$\phi(b + \varepsilon) - \phi(b) \approx \sum_{j=1}^m \frac{\partial \phi}{\partial b_j}(b) \varepsilon_j.$$

The quantity $\partial\phi/\partial b_j$ is the marginal value to this factory of one more unit of material j : it is called the **shadow price** of material j . It is a ‘shadow’ price because it is invisible from outside – it depends entirely on the internal state of the factory, not on the market.

Theorem 2.14

If ϕ is differentiable at b and has a supporting hyperplane at b with normal λ , then $\lambda = \nabla\phi(b)$.

Proof. Let $a \in \mathbb{R}^m$ be arbitrary and $\delta > 0$. The supporting hyperplane condition at $c = b + \delta a$ gives

$$\phi(b + \delta a) - \phi(b) \geq \delta \lambda \cdot a, \quad \text{so} \quad \frac{\phi(b + \delta a) - \phi(b)}{\delta} \geq \lambda \cdot a.$$

Letting $\delta \rightarrow 0$ and using differentiability, $\nabla\phi(b) \cdot a \geq \lambda \cdot a$. Since a was arbitrary we may also apply this with $-a$, giving $\nabla\phi(b) \cdot a = \lambda \cdot a$ for all a , and hence $\lambda = \nabla\phi(b)$. $\square$

Example 2.15

Let us verify this in a case where we can compute everything. Consider

$$\text{minimize } \frac{1}{2}(x_1^2 + x_2^2) \text{ subject to } x_1 + x_2 = b,$$

over $x \in \mathbb{R}^2$. The Lagrangian is

$$L(x, \lambda) = \frac{1}{2}(x_1^2 + x_2^2) - \lambda(x_1 + x_2 - b) = \left(\frac{1}{2}x_1^2 - \lambda x_1\right) + \left(\frac{1}{2}x_2^2 - \lambda x_2\right) + \lambda b,$$

which is bounded below for every λ , so $\Lambda = \mathbb{R}$. Minimizing each bracket gives $x_1 = x_2 = \lambda$, and feasibility then forces $2\lambda = b$, so

$$\lambda^* = \frac{b}{2}, \quad x^* = \left(\frac{b}{2}, \frac{b}{2}\right), \quad \phi(b) = \frac{b^2}{4}.$$

And indeed $\phi'(b) = b/2 = \lambda^*$, exactly as the theorem promises. Concretely: relaxing the constraint from b to $b + \varepsilon$ increases the optimal cost by about $\lambda^*\varepsilon$.

So the Lagrange multipliers *are* the shadow prices. This makes complementary slackness completely intuitive: if a raw material is not fully used up ($s_j > 0$, the constraint is slack) then an extra unit of it is worth nothing, so its shadow price λ_j is zero. Conversely, if it has a nonzero price then we must be using every last unit of it.

There is a matching interpretation of the dual problem. Think of it from the supplier’s point of view. The supplier posts prices λ for the raw materials and offers to buy the finished products. Given prices λ , the factory owner will choose x to maximize

$$\underbrace{f(x)}_{\text{revenue}} - \underbrace{\lambda \cdot (h(x) - b)}_{\text{cost of materials}},$$

which is exactly the Lagrangian. The supplier, knowing this, chooses λ to make the factory owner’s best achievable profit as small as possible – that is, the supplier solves the dual problem. Weak duality says the supplier can never squeeze the owner below the true optimum, and strong duality says that with the right prices the two coincide: at equilibrium there is no advantage to either party.

§3 Linear Programming

We now specialize to the case where everything in sight is linear. This may sound like a drastic restriction, and in a sense it is, but linear programs are astonishingly expressive – an enormous fraction of the optimization problems that actually arise in industry are linear programs, or are usefully approximated by them. More importantly for us, in this setting the general theory of the previous section becomes completely explicit and computable, and we will be able to write down an algorithm that solves any linear program by hand.

§3.1 Linear Programs

Definition 3.1 (Linear Program)

A **linear program** is an optimization problem in which the objective function and all of the constraints are linear (more precisely, affine) functions of the decision variables.

A general linear program might look something like

$$\begin{aligned} \text{minimize} \quad & 2x_1 - x_2 + 4x_3 \\ \text{subject to} \quad & x_1 + x_2 + x_4 \leq 2, \\ & 3x_2 - x_3 = 5, \\ & x_3 + x_4 \geq 3, \\ & x_1 \geq 0, \ x_3 \leq 0, \end{aligned}$$

with x_2 and x_4 unconstrained in sign. This is a mess of different constraint types, and before doing anything we would like to tidy it up. Happily, all of the constraint types are interchangeable.

Definition 3.2 (Equivalent Problems)

Two optimization problems are **equivalent** if from a feasible solution to either one we can construct a feasible solution to the other with the same cost. Rewriting a problem as an equivalent one is called **reduction**.

The reductions we need are all elementary:

- $a \cdot x \leq b$ becomes $-a \cdot x \geq -b$;
- $a \cdot x \geq b$ becomes $a \cdot x - s = b$ with $s \geq 0$ (a *surplus* variable), and $a \cdot x \leq b$ becomes $a \cdot x + s = b$ with $s \geq 0$ (a *slack* variable);
- $a \cdot x = b$ becomes the pair $a \cdot x \geq b$ and $-a \cdot x \geq -b$;
- a variable $x_j \leq 0$ is replaced by $-x_j \geq 0$;
- a variable x_j of unrestricted sign is replaced by $x_j^+ - x_j^-$ with $x_j^+, x_j^- \geq 0$.

Applying these as needed, we can put any linear program into either of two standard shapes.

Definition 3.3 (General and Standard Form)

A linear program is in **general form** if it is written

$$\begin{array}{ll} \text{minimize} & c \cdot x \\ \text{subject to} & Ax \geq b, \end{array}$$

and in **standard form** if it is written

$$\begin{array}{ll} \text{minimize} & c \cdot x \\ \text{subject to} & Ax = b, \\ & x \geq 0, \end{array}$$

where in both cases $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$.

Standard form is what we will use for computation, since the constraint $Ax = b$ is just a linear system; general form is convenient for stating duality. Both are fully general: to get from general to standard form, first introduce surplus variables to turn $Ax \geq b$ into $Ax - s = b$ with $s \geq 0$, then split each unrestricted x_j as $x_j^+ - x_j^-$, giving

$$\begin{array}{ll} \text{minimize} & c \cdot x^+ - c \cdot x^- \\ \text{subject to} & Ax^+ - Ax^- - s = b, \\ & x^+, x^-, s \geq 0, \end{array}$$

which is in standard form in the variables $(x^+, x^-, s) \in \mathbb{R}_{\geq 0}^{2n+m}$.

Notice that reduction to standard form typically increases the number of variables. That is fine: what matters is that the constraint matrix stays sparse and the structure stays linear.

§3.2 A Two-Dimensional Example

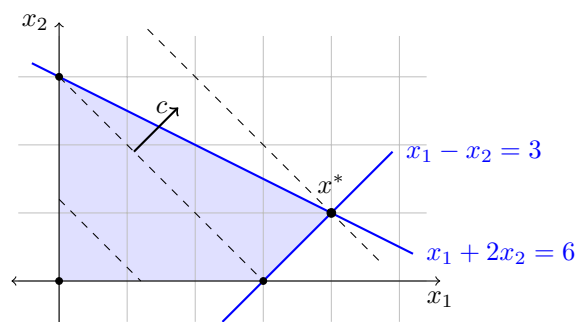
Before doing any theory, it is worth solving a small linear program by drawing a picture, because the picture explains everything that follows.

Example 3.4

Consider the problem

$$\begin{array}{ll} \text{maximize} & x_1 + x_2 \\ \text{subject to} & x_1 + 2x_2 \leq 6, \\ & x_1 - x_2 \leq 3, \\ & x_1, x_2 \geq 0. \end{array}$$

The feasible set is the shaded polygon below: the intersection of four half-planes. The dashed lines are the level sets $x_1 + x_2 = \text{const}$ of the objective, which all have gradient $(1, 1)$. To maximize the objective, we slide the dashed line in the direction $(1, 1)$ as far as we can while still touching the feasible set.



The optimum is at $x^* = (4, 1)$, where the two constraint lines meet, with optimal value 5. Two features of this picture are completely general, and are the theoretical backbone of linear programming:

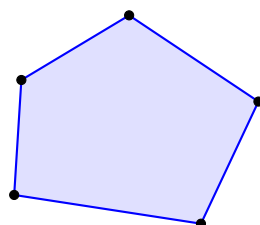
- (i) the feasible set is a *polyhedron* – an intersection of finitely many half-spaces; and
- (ii) the optimum is attained at a *corner* of that polyhedron.

If the objective happens to be parallel to one of the edges, then a whole edge is optimal – but even then, a corner is among the optimal points. So to solve a linear program, it suffices to search the (finitely many!) corners. Our next job is to say precisely what a corner is, algebraically.

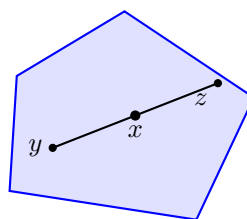
§3.3 Extreme Points

Definition 3.5 (Extreme Point)

A point x in a convex set C is an **extreme point** if it cannot be written as $x = \delta y + (1 - \delta)z$ for $y, z \in C$ with $y \neq z$ and $\delta \in (0, 1)$. That is, x does not lie strictly inside any line segment contained in C .



extreme points



x is not extreme

The relevance is the following general fact, which explains observation (ii) above.

Proposition 3.6

Let f be a convex function on a convex set C , and let $y, z \in C$. Then f attains its maximum over the segment $[y, z]$ at an endpoint. Consequently, if C is a bounded polyhedron then $\max_{x \in C} f(x)$ is attained at an extreme point of C .

Proof. For the first part, if $x = \delta y + (1 - \delta)z$ then by convexity

$$f(x) \leq \delta f(y) + (1 - \delta)f(z) \leq \max\{f(y), f(z)\}.$$

For the second, every point of a bounded polyhedron C can be written as a convex

combination $x = \sum_{i=1}^k \delta_i x^{(i)}$ of extreme points $x^{(i)}$, with $\delta_i \geq 0$ and $\sum \delta_i = 1$. By convexity again,

$$f(x) \leq \sum_{i=1}^k \delta_i f(x^{(i)}) \leq \max_i f(x^{(i)}),$$

so no interior point beats the best extreme point. $\square$

Since a linear function is convex (and concave), the maximum *and* the minimum of a linear function on a bounded polyhedron are both attained at extreme points. So the corners really are all we need to look at. What we lack is a way of finding them without drawing a picture, which is hopeless in $\mathbb{R}^{100}$.

§3.4 Basic Solutions

Consider now a linear program in standard form,

$$\begin{aligned} &\text{minimize} && c \cdot x \\ &\text{subject to} && Ax = b, \\ &&& x \geq 0, \end{aligned}$$

with $A \in \mathbb{R}^{m \times n}$ and typically $n > m$, so that $Ax = b$ is an underdetermined system with a whole affine space of solutions. Intuitively, a corner of the feasible set is a point where as many constraints as possible are tight; since the only inequality constraints are $x_j \geq 0$, this means a point with as many zero coordinates as possible.

Definition 3.7 (Basic Solution)

A vector $x \in \mathbb{R}^n$ is a **basic solution** of $Ax = b$ if $Ax = b$ and x has at most m nonzero entries. If in addition $x \geq 0$, it is a **basic feasible solution** (bfs).

To construct basic solutions, we choose which m coordinates are allowed to be nonzero. We make two standing assumptions on A , both harmless.

- (A) The m rows of A are linearly independent. (Otherwise some constraint is redundant or inconsistent, and we can delete it.)
- (B) Every set of m columns of A is linearly independent. (This is a genericity assumption; we can always perturb b slightly to arrange it.)

Given a choice of indices $B = \{B(1), \dots, B(m)\} \subseteq \{1, \dots, n\}$, write A_j for the j th column of A and let

$$A_B = (A_{B(1)} \quad \cdots \quad A_{B(m)}) \in \mathbb{R}^{m \times m}$$

be the corresponding **basis matrix**. Setting $x_j = 0$ for $j \notin B$, the equation $Ax = b$ reduces to $A_B x_B = b$, where $x_B = (x_{B(1)}, \dots, x_{B(m)})$. By assumption (B), A_B is invertible, and so

$$x_B = A_B^{-1} b.$$

The indices in B are the **basic indices**, the corresponding variables are the **basic variables**, and the rest are the **non-basic variables** (all set to zero). If moreover $A_B^{-1} b \geq 0$, we have found a basic feasible solution.

- (C) (**Non-degeneracy**) Every basic solution has *exactly* m nonzero entries.

This last assumption is not without loss of generality, and we will discuss what goes wrong without it in Section ?? . Until then we assume it.

Now we can connect the algebra to the geometry.

Theorem 3.8 (Basic Feasible Solutions are Extreme Points)

Let $P = \{x : Ax = b, x \geq 0\}$. Then x is an extreme point of P if and only if x is a basic feasible solution.

Proof. Suppose first that x is a bfs, with basis B , and suppose $x = \delta y + (1 - \delta)z$ with $y, z \in P$ and $\delta \in (0, 1)$. For $j \notin B$ we have $x_j = 0$, so $\delta y_j + (1 - \delta)z_j = 0$; but $y_j, z_j \geq 0$ and $\delta \in (0, 1)$, so $y_j = z_j = 0$. Hence y and z are also supported on B , and $A_B y_B = b = A_B z_B$. As A_B is invertible, $y_B = z_B = A_B^{-1}b = x_B$, and so $y = z = x$. Thus x is extreme.

Conversely suppose $x \in P$ is not a bfs, so x has $r > m$ nonzero entries, say in positions $i_1, \dots, i_r$. The columns $A_{i_1}, \dots, A_{i_r}$ are $r > m$ vectors in $\mathbb{R}^m$, hence linearly dependent: there are weights $w_{i_1}, \dots, w_{i_r}$, not all zero, with

$$w_{i_1}A_{i_1} + \dots + w_{i_r}A_{i_r} = 0.$$

Extend w to a vector in $\mathbb{R}^n$ by setting $w_i = 0$ for $i \notin \{i_1, \dots, i_r\}$. Then $w \neq 0$ and $Aw = 0$, so $A(x \pm \varepsilon w) = b$ for every ε . Moreover w is supported where x is strictly positive, so for small enough $\varepsilon > 0$ we have $x \pm \varepsilon w \geq 0$, i.e. $x \pm \varepsilon w \in P$. But then x is the midpoint of these two distinct points of P , so x is not extreme. $\square$

Corollary 3.9

There are at most $\binom{n}{m}$ basic solutions, and hence at most $\binom{n}{m}$ extreme points of P .

Example 3.10

Let us see this explicitly on the example of Section 3.2. Writing it as a minimization and adding slack variables x_3, x_4 , the standard form is: minimize $-x_1 - x_2$ subject to

$$x_1 + 2x_2 + x_3 = 6, \quad x_1 - x_2 + x_4 = 3, \quad x \geq 0,$$

so $m = 2$, $n = 4$ and there are at most $\binom{4}{2} = 6$ basic solutions. Let us list them all.

basis	$x = (x_1, x_2, x_3, x_4)$	feasible?	cost
$\{3, 4\}$	$(0, 0, 6, 3)$	yes	0
$\{1, 3\}$	$(3, 0, 3, 0)$	yes	-3
$\{1, 4\}$	$(6, 0, 0, -3)$	no	—
$\{2, 3\}$	$(0, -3, 12, 0)$	no	—
$\{2, 4\}$	$(0, 3, 0, 6)$	yes	-3
$\{1, 2\}$	$(4, 1, 0, 0)$	yes	-5

For instance, for the basis $\{1, 2\}$ we set $x_3 = x_4 = 0$ and solve $x_1 + 2x_2 = 6$, $x_1 - x_2 = 3$, giving $(x_1, x_2) = (4, 1)$. Comparing with the picture in Section 3.2, the four basic feasible solutions are exactly the four corners $(0, 0)$, $(3, 0)$, $(0, 3)$ and $(4, 1)$ of the polygon, and the two infeasible basic solutions are the points $(6, 0)$ and $(0, -3)$

where two constraint lines meet *outside* the feasible region. The best cost among the feasible ones is -5 at $(4, 1)$, in agreement with what we found geometrically.

Theorem 3.11 (Fundamental Theorem of Linear Programming)

If a linear program in standard form is feasible and bounded, then there is an optimal solution which is a basic feasible solution.

Proof. Let x be an optimal solution with as few nonzero entries as possible, say r of them. If $r \leq m$ then x is a bfs and we are done, so suppose $r > m$. By the argument in the previous proof, there is $w \neq 0$ with $Aw = 0$, supported on the positions where x is nonzero.

Consider $x_\theta = x + \theta w$, which satisfies $Ax_\theta = b$ for all $\theta \in \mathbb{R}$, and lies in P for all θ in some interval around 0. Its cost is $c \cdot x + \theta(c \cdot w)$. If $c \cdot w \neq 0$ then choosing the sign of θ appropriately strictly decreases the cost, contradicting optimality of x . So $c \cdot w = 0$ and x_θ is optimal for every feasible θ .

Now increase $|\theta|$ (in either direction) until some coordinate of x_θ first hits zero – this happens for a finite θ since w is nonzero and supported where x is. The resulting point is optimal, feasible, and has strictly fewer than r nonzero entries, contradicting minimality of r . $\square$

This is genuinely good news: it reduces an optimization over a continuum to a search over a finite set. The bad news is that $\binom{n}{m}$ is astronomically large – for $n = 100$ and $m = 50$ it exceeds 10^{29} – so brute force is out of the question. The simplex method, which we come to shortly, is a clever way of walking from one basic feasible solution to a better one.

§3.5 Duality for Linear Programs

Before the algorithm, let us work out what the duality theory of Section 2 says here. It says something extremely concrete.

Consider a linear program in general form, with surplus variables added:

$$\begin{aligned} &\text{minimize} && c \cdot x \\ &\text{subject to} && Ax - s = b, \\ &&& s \geq 0, \end{aligned}$$

where x is unrestricted. The Lagrangian is

$$L(x, s, \lambda) = c \cdot x - \lambda \cdot (Ax - s - b) = (c - A^T \lambda) \cdot x + \lambda \cdot s + \lambda \cdot b.$$

Since x ranges over all of $\mathbb{R}^n$, this is bounded below only if $c - A^T \lambda = 0$; and since s ranges over $[0, \infty)^m$, we need $\lambda \geq 0$. Given both, the infimum is attained at $s = 0$ and equals $\lambda \cdot b$. Hence

$$\Lambda = \{\lambda : A^T \lambda = c, \lambda \geq 0\}, \quad g(\lambda) = \lambda \cdot b,$$

and we have proved the following.

Proposition 3.12 (Dual of a General-Form LP)

The dual of

$$\text{minimize } c \cdot x \quad \text{subject to } Ax \geq b$$

is

$$\text{maximize } \lambda \cdot b \quad \text{subject to } A^T \lambda = c, \lambda \geq 0.$$

Notice that the dual of a general-form problem is in standard form. The same computation for a standard-form primal gives the reverse.

Proposition 3.13 (Dual of a Standard-Form LP)

The dual of

$$\text{minimize } c \cdot x \quad \text{subject to } Ax = b, x \geq 0$$

is

$$\text{maximize } \lambda \cdot b \quad \text{subject to } A^T \lambda \leq c.$$

Proof. Here $X = \{x : x \geq 0\}$ and $L(x, \lambda) = c \cdot x - \lambda \cdot (Ax - b) = (c - A^T \lambda) \cdot x + \lambda \cdot b$. Since x ranges over the non-negative orthant, this is bounded below exactly when $c - A^T \lambda \geq 0$, and then the infimum is attained at $x = 0$ and equals $\lambda \cdot b$. $\square$

These two facts fit together neatly.

Proposition 3.14

The dual of the dual is the primal.

Proof. Start with the standard-form primal and its dual ‘maximize $\lambda \cdot b$ subject to $A^T \lambda \leq c$ ’. Rewrite this as a minimization in general form: putting $\mu = -\lambda$,

$$\text{minimize } \mu \cdot b \quad \text{subject to } A^T \mu \geq -c,$$

whose optimal value is the negative of the original. Applying the general-form dual with data $(b, A^T, -c)$ in place of (c, A, b) , the dual is

$$\text{maximize } -\theta \cdot c \quad \text{subject to } A\theta = b, \theta \geq 0,$$

that is, minimize $c \cdot \theta$ subject to $A\theta = b, \theta \geq 0$, which is the original primal. $\square$

For a completely general linear program, the pattern of the dual is worth recording; each of these can be derived by the reductions above.

Proposition 3.15 (Dual of a General Linear Program)

The dual of

$$\begin{aligned} &\text{minimize } c \cdot x \\ &\text{subject to } a_i \cdot x \geq b_i \quad i \in M_1, \\ &\quad \quad \quad a_i \cdot x \leq b_i \quad i \in M_2, \end{aligned}$$

is

$$\begin{aligned}
 & a_i \cdot x = b_i \quad i \in M_3, \\
 & x_j \geq 0 \quad j \in N_1, \\
 & x_j \leq 0 \quad j \in N_2, \\
 & x_j \text{ free} \quad j \in N_3,
 \end{aligned}$$

$$\begin{aligned}
 & \text{maximize} \quad \lambda \cdot b \\
 & \text{subject to} \quad \lambda_i \geq 0 \quad i \in M_1, \\
 & \quad \quad \quad \lambda_i \leq 0 \quad i \in M_2, \\
 & \quad \quad \quad \lambda_i \text{ free} \quad i \in M_3, \\
 & \quad \quad \quad \lambda \cdot A_j \leq c_j \quad j \in N_1, \\
 & \quad \quad \quad \lambda \cdot A_j \geq c_j \quad j \in N_2, \\
 & \quad \quad \quad \lambda \cdot A_j = c_j \quad j \in N_3.
 \end{aligned}$$

The mnemonic is that constraints in the primal correspond to variables in the dual and vice versa. It is worth having the correspondence in a table:

primal (minimize)	dual (maximize)
i th constraint $\geq$	i th variable ≥ 0
i th constraint $\leq$	i th variable ≤ 0
i th constraint $=$	i th variable free
j th variable ≥ 0	j th constraint $\leq$
j th variable ≤ 0	j th constraint $\geq$
j th variable free	j th constraint $=$
objective vector c	right-hand side c
right-hand side b	objective vector b
constraint matrix A	constraint matrix A^T

There is an easy way to remember the direction of the inequalities: whichever choice makes weak duality come out right is the correct one. For instance in the standard-form case, a primal variable $x_j \geq 0$ contributes $c_j x_j$ to the objective, and we need $(c_j - \lambda \cdot A_j)x_j \geq 0$ for the bound $c \cdot x \geq \lambda \cdot b$ to work; with $x_j \geq 0$ this forces $\lambda \cdot A_j \leq c_j$, which is a $\leq$ constraint in the dual.

Since a linear program has a convex objective, convex constraints and convex feasible region, the theorem of Section 2.4 applies immediately.

Theorem 3.16 (Strong Duality for Linear Programs)

If a linear program is feasible and bounded, then so is its dual, and their optimal values are equal.

Proof. The value function $\phi(d) = \inf\{c \cdot x : Ax \geq d\}$ is convex by the theorem in Section 2.4 (taking $X = \mathbb{R}^n$, and noting that $f(x) = c \cdot x$ and $h(x) = -Ax$ are linear, hence convex). Hence ϕ has a supporting hyperplane at b , and strong duality holds. $\square$

§3.6 Complementary Slackness for Linear Programs

Strong duality gives us a certificate of optimality, and in the linear case that certificate takes an extremely usable form.

Theorem 3.17 (Optimality Conditions)

Let x be feasible for the primal and λ feasible for the dual, in the general setting of the previous proposition. Then x and λ are both optimal if and only if

- (i) $\lambda_i(a_i \cdot x - b_i) = 0$ for all i , and
- (ii) $(c_j - \lambda \cdot A_j)x_j = 0$ for all j .

Proof. Set $u_i = \lambda_i(a_i \cdot x - b_i)$ and $v_j = (c_j - \lambda \cdot A_j)x_j$. Feasibility of x and λ makes each $u_i \geq 0$ and each $v_j \geq 0$: for instance if $i \in M_1$ then $a_i \cdot x - b_i \geq 0$ and $\lambda_i \geq 0$, while if $i \in M_2$ both factors flip sign, and if $i \in M_3$ the second factor is zero. The same case analysis works for v_j . Now

$$\sum_i u_i = \lambda \cdot (Ax) - \lambda \cdot b, \quad \sum_j v_j = c \cdot x - \lambda \cdot (Ax),$$

so adding,

$$0 \leq \sum_i u_i + \sum_j v_j = c \cdot x - \lambda \cdot b.$$

If (i) and (ii) hold then all u_i, v_j vanish, so $c \cdot x = \lambda \cdot b$, and by weak duality both must be optimal. Conversely if both are optimal then strong duality gives $c \cdot x = \lambda \cdot b$, so $\sum u_i + \sum v_j = 0$; as all the terms are non-negative, they must all be zero. $\square$

Conditions (i) and (ii) are the **complementary slackness** conditions for a linear program. Condition (i) says: if the dual variable λ_i is nonzero then the i th primal constraint is tight. Condition (ii) says: if the primal variable x_j is nonzero then the j th dual constraint is tight. So the whole state of the optimum is captured by:

$$\text{primal feasibility} \quad + \quad \text{dual feasibility} \quad + \quad \text{complementary slackness}.$$

These three conditions are what the simplex method will maintain and enforce, and the algorithm can be understood entirely in terms of them: it keeps the first and third true at all times, and works to make the second true.

Example 3.18

Return to our two-dimensional example, written as a minimization in general form. We minimize $-x_1 - x_2$ subject to

$$\begin{pmatrix} -1 & -2 \\ -1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \geq \begin{pmatrix} -6 \\ -3 \\ 0 \\ 0 \end{pmatrix}.$$

The dual is: maximize $-6\lambda_1 - 3\lambda_2$ subject to $\lambda \geq 0$ and

$$-\lambda_1 - \lambda_2 + \lambda_3 = -1, \quad -2\lambda_1 + \lambda_2 + \lambda_4 = -1.$$

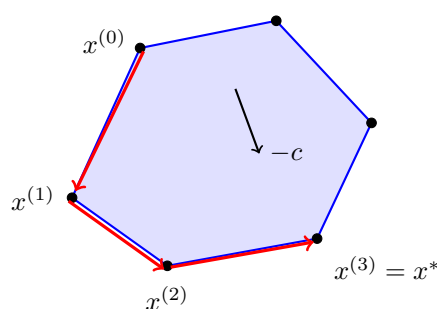
At the primal optimum $x^* = (4, 1)$ the first two constraints are tight and the last two are slack, so complementary slackness forces $\lambda_3 = \lambda_4 = 0$. The remaining equations give $\lambda_1 + \lambda_2 = 1$ and $2\lambda_1 - \lambda_2 = 1$, so $\lambda = (2/3, 1/3, 0, 0)$, which is indeed

non-negative. The dual objective is $-6 \cdot \frac{2}{3} - 3 \cdot \frac{1}{3} = -5$, matching the primal optimal value of -5 . So each certifies the other.

The shadow price interpretation is now concrete: $\lambda_1 = 2/3$ says that relaxing the first constraint from $x_1 + 2x_2 \leq 6$ to ≤ 7 would increase the maximum of $x_1 + x_2$ by $2/3$.

§4 The Simplex Method

We know that an optimal solution can be found among the basic feasible solutions, and we know that there are far too many of these to enumerate. The **simplex method**, due to Dantzig, resolves this by moving from one basic feasible solution to an adjacent one, always decreasing the cost, until no improving move exists. Geometrically, it walks along the edges of the feasible polyhedron from corner to corner, downhill all the way.



§4.1 Reduced Costs

To turn the picture into an algorithm we need to answer two questions: when is a basic feasible solution optimal, and if it is not, which neighbouring solution should we move to? Both answers come from duality.

Suppose we are at a basic feasible solution x with basis matrix $B = A_B$ (we will write B for both the index set and the matrix, as is traditional). We have primal feasibility for free. Let us try to construct a λ making complementary slackness hold, and then check whether it is dual feasible.

For a standard-form problem, the three optimality conditions read

- (primal feasibility) $Ax = b, x \geq 0$;
- (dual feasibility) $A^T \lambda \leq c$;
- (complementary slackness) $x \cdot (c - A^T \lambda) = 0$.

Since $x_j = 0$ for non-basic j , complementary slackness only constrains the basic coordinates, where $x_B > 0$; so it forces $c_B - B^T \lambda = 0$, that is

$$\lambda = (B^T)^{-1} c_B.$$

With this choice of λ , primal feasibility and complementary slackness both hold. Only dual feasibility $A^T \lambda \leq c$ remains, and this motivates the following definition.

Definition 4.1 (Reduced Costs)

Given a basis B , the vector of **reduced costs** is

$$\bar{c} = c - A^T(B^T)^{-1}c_B, \quad \text{componentwise} \quad \bar{c}_j = c_j - c_B \cdot (B^{-1}A_j).$$

By construction, dual feasibility is precisely the statement $\bar{c} \geq 0$, so we have proved most of the following.

Theorem 4.2 (Optimality Criterion)

Let x be a basic feasible solution with basis B and reduced cost vector $\bar{c}$. If $\bar{c} \geq 0$ then x is optimal.

Note that $\bar{c}_j = 0$ for every basic j , since $B^{-1}A_{B(i)} = e_i$ and $c_B \cdot e_i = c_{B(i)}$. So we only need to inspect the non-basic reduced costs.

The reduced costs have a direct meaning too, which tells us what to do when some of them are negative. Suppose j is non-basic and we try to increase x_j from 0 while keeping $Ax = b$. Formally, define the j th **basic direction** d by $d_j = 1$, $d_i = 0$ for all other non-basic i , and d_B determined by the requirement $Ad = 0$:

$$Bd_B + A_j = 0 \implies d_B = -B^{-1}A_j.$$

Then $A(x + \theta d) = b$ for all θ , and since $x_B > 0$ (non-degeneracy!) we have $x + \theta d \geq 0$ for all small enough $\theta > 0$. So d is a feasible direction. Its effect on the cost is

$$c \cdot (x + \theta d) = c \cdot x + \theta (c_j - c_B \cdot (B^{-1}A_j)) = c \cdot x + \theta \bar{c}_j.$$

So $\bar{c}_j$ is exactly the rate of change of the cost as we bring x_j into the solution. If $\bar{c}_j < 0$, moving in the j th basic direction reduces the cost, and we should do so as far as possible.

How far can we go? We need $x + \theta d \geq 0$. The non-basic coordinates are fine (x_j increases from 0, the others stay at 0). For the basic coordinates, writing $u = -d_B = B^{-1}A_j$, we need $x_{B(i)} - \theta u_i \geq 0$ for each i . If $u \leq 0$ this holds for all $\theta > 0$, and the cost is unbounded below. Otherwise the largest permissible step is

$$\theta^* = \min_{\{i: u_i > 0\}} \frac{x_{B(i)}}{u_i},$$

the so-called **ratio test**. Let ℓ attain the minimum. At $\theta = \theta^*$ the basic variable $x_{B(\ell)}$ becomes zero, so we have arrived at a new point with (again) m nonzero entries: x_j has entered the basis and $x_{B(\ell)}$ has left it.

Theorem 4.3

With the notation above, $y = x + \theta^*d$ is a basic feasible solution with basis obtained from B by replacing $A_{B(\ell)}$ with A_j , and $c \cdot y = c \cdot x + \theta^*\bar{c}_j < c \cdot x$.

Proof. We have $Ay = b$ and $y \geq 0$ by the choice of θ^* , and $y_{B(\ell)} = 0$ while $y_j = \theta^* > 0$, so exactly m entries are nonzero and they sit in the claimed positions. The cost formula was computed above, and $\theta^* > 0$, $\bar{c}_j < 0$ give the strict decrease. $\square$

Putting this together gives the algorithm.

Definition 4.4 (The Simplex Method)

To minimize $c \cdot x$ subject to $Ax = b$, $x \geq 0$:

- (1) Start at a basic feasible solution x with basis B .

- (2) Compute the reduced costs $\bar{c}$. If $\bar{c} \geq 0$, stop: x is optimal.
- (3) Otherwise choose j with $\bar{c}_j < 0$, and set $u = B^{-1}A_j$.
- (4) If $u \leq 0$, stop: the problem is unbounded.
- (5) Otherwise compute $\theta^* = \min\{x_{B(i)}/u_i : u_i > 0\}$, attained at $i = \ell$.
- (6) Replace $B(\ell)$ by j in the basis, update $x \mapsto x + \theta^*d$, and go to (2).

Since the cost strictly decreases at each step (under the non-degeneracy assumption) we never revisit a basis, and there are finitely many bases, so the algorithm terminates.

§4.2 The Simplex Tableau

Executing the algorithm as stated would require inverting B at every step, which is intolerable by hand. The **tableau** is a bookkeeping device that maintains all of the required quantities automatically using row operations.

Definition 4.5 (Simplex Tableau)

The **simplex tableau** for the basis B is the array

$-c_B \cdot x_B$	$\bar{c}^T$
$B^{-1}b$	$B^{-1}A$

or in more detail

$-c_B \cdot x_B$	$\bar{c}_1$	$\bar{c}_2$	$\cdots$	$\bar{c}_n$
$x_{B(1)}$	$\vdots$	$\vdots$		$\vdots$
$\vdots$	$B^{-1}A_1$	$B^{-1}A_2$	$\cdots$	$B^{-1}A_n$
$x_{B(m)}$	$\vdots$	$\vdots$		$\vdots$

So the top-left entry is minus the current cost, the top row holds the reduced costs, the left column holds the current basic feasible solution, and the body holds $B^{-1}A$ – which is what the ratio test and basic directions need. Note that the basic columns of $B^{-1}A$ form an identity matrix, and the corresponding entries of the top row are zero; this is how we recognize the basis by looking at the tableau.

The remarkable fact – and it really is worth pausing over – is that the update in step (6) of the algorithm is achieved by pure Gaussian elimination on the tableau. The recipe is:

- (i) Choose a **pivot column** j with $\bar{c}_j < 0$.
- (ii) Choose the **pivot row** ℓ minimizing $x_{B(i)}/u_i$ among rows with $u_i > 0$, where u_i is the entry in row i of column j . The entry at (ℓ, j) is the **pivot**.
- (iii) Divide row ℓ by the pivot, then add multiples of row ℓ to every other row (including row 0) so that column j becomes the standard basis vector e_ℓ .

Why this works is not hard to see: multiplying on the left by $\bar{B}^{-1}B$ is exactly a sequence of row operations, and the row operations that turn column j into e_ℓ are precisely those which implement the change of basis.

Example 4.6

We solve

$$\begin{aligned}
 &\text{minimize} && -x_1 - x_2 - x_3 \\
 &\text{subject to} && x_1 + 2x_2 + 2x_3 \leq 10, \\
 &&& 2x_1 + x_2 + 2x_3 \leq 10, \\
 &&& 2x_1 + 2x_2 + x_3 \leq 20, \\
 &&& x_1, x_2, x_3 \geq 0.
 \end{aligned}$$

Adding slack variables $x_4, x_5, x_6 \geq 0$ puts this in standard form with

$$A = \begin{pmatrix} 1 & 2 & 2 & 1 & 0 & 0 \\ 2 & 1 & 2 & 0 & 1 & 0 \\ 2 & 2 & 1 & 0 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 10 \\ 10 \\ 20 \end{pmatrix}, \quad c = (-1, -1, -1, 0, 0, 0).$$

The slack variables give us a basic feasible solution for free: take $B = \{4, 5, 6\}$, so B is the identity matrix and $x = (0, 0, 0, 10, 10, 20)$. Since $c_B = 0$ we have $\bar{c} = c$ and cost 0, and the initial tableau is

0	-1	-1	-1	0	0	0
10	1	2	2	1	0	0
10	2	1	2	0	1	0
20	2	2	1	0	0	1

Since $\bar{c}_1 = -1 < 0$ we pivot in column 1. The ratios are $10/1 = 10$, $10/2 = 5$ and $20/2 = 10$, so the smallest is in row 2 and the pivot is the boxed 2. We perform

$$R_2 \mapsto \frac{1}{2}R_2, \quad R_0 \mapsto R_0 + \frac{1}{2}R_2^{\text{old}}, \quad R_1 \mapsto R_1 - \frac{1}{2}R_2^{\text{old}}, \quad R_3 \mapsto R_3 - R_2^{\text{old}},$$

obtaining

5	0	$-\frac{1}{2}$	0	0	$\frac{1}{2}$	0
5	0	$\frac{3}{2}$	1	1	$-\frac{1}{2}$	0
5	1	$\frac{1}{2}$	1	0	$\frac{1}{2}$	0
10	0	1	-1	0	-1	1

The basis is now $\{4, 1, 6\}$ and the current solution is $x = (5, 0, 0, 5, 0, 10)$ with cost -5 (remember the top-left entry is *minus* the cost).

Now $\bar{c}_2 = -\frac{1}{2} < 0$, so we pivot in column 2. The ratios are $5/\frac{3}{2} = \frac{10}{3}$, $5/\frac{1}{2} = 10$ and $10/1 = 10$, so we pivot on the boxed $\frac{3}{2}$ in row 1. We perform

$$R_1 \mapsto \frac{2}{3}R_1, \quad R_0 \mapsto R_0 + \frac{1}{3}R_1^{\text{old}}, \quad R_2 \mapsto R_2 - \frac{1}{3}R_1^{\text{old}}, \quad R_3 \mapsto R_3 - \frac{2}{3}R_1^{\text{old}},$$

giving

$\frac{20}{3}$	0	0	$\frac{1}{3}$	$\frac{1}{3}$	$\frac{1}{3}$	0
$\frac{10}{3}$	0	1	$\frac{2}{3}$	$\frac{2}{3}$	$-\frac{1}{3}$	0
$\frac{10}{3}$	1	0	$\frac{2}{3}$	$-\frac{1}{3}$	$\frac{2}{3}$	0
$\frac{20}{3}$	0	0	$-\frac{5}{3}$	$-\frac{2}{3}$	$-\frac{2}{3}$	1

Every reduced cost is now non-negative, so we are optimal. Reading off the basis $\{2, 1, 6\}$ and the left-hand column, the optimal solution is

$$x^* = \left(\frac{10}{3}, \frac{10}{3}, 0, 0, 0, \frac{20}{3}\right)$$

with optimal cost $-\frac{20}{3}$.

We get the dual solution for free as well: the reduced costs of the slack variables are $\bar{c}_4 = \bar{c}_5 = \frac{1}{3}$, $\bar{c}_6 = 0$, and since $\bar{c}_{n+i} = -\lambda_i$ for slack variable i when the slack columns start as the identity, we read off $\lambda = (-\frac{1}{3}, -\frac{1}{3}, 0)$. The third constraint has zero multiplier, which is consistent with the fact that $x_6 = \frac{20}{3} > 0$ – that constraint is slack at the optimum.

Example 4.7 (An Unbounded Problem)

Consider the problem: minimize $-x_1 - x_2$ subject to $x_1 - x_2 \leq 1$ and $x_1, x_2 \geq 0$. Adding a slack variable, the standard form has

$$A = \begin{pmatrix} 1 & -1 & 1 \end{pmatrix}, \quad b = (1), \quad c = (-1, -1, 0),$$

and the basis $B = \{3\}$ gives the initial tableau

0	-1	-1	0
1	1	-1	1

Both $\bar{c}_1$ and $\bar{c}_2$ are negative. If we choose the pivot column $j = 2$, then $u = B^{-1}A_2 = (-1) \leq 0$, so no ratio test is possible and the problem is unbounded. Sure enough, $x = (t, t, 1)$ is feasible for every $t \geq 0$ and has cost $-2t \rightarrow -\infty$: the feasible region contains the ray in the direction $(1, 1)$, along which the objective decreases forever.

Note that had we chosen the pivot column $j = 1$ instead, we would have pivoted (with $\theta^* = 1$) to the basis $\{1\}$ and only discovered the unboundedness at the next step. Unboundedness is a property of the problem, not of the pivoting rule; the algorithm will always find it eventually.

§4.3 Degeneracy and Bland's Rule

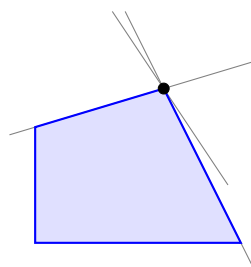
We have been assuming non-degeneracy throughout, and it is time to see what it was doing.

Definition 4.8 (Degeneracy)

A basic feasible solution is **degenerate** if it has fewer than m nonzero entries, so that some basic variable takes the value zero.

Geometrically, degeneracy happens when more than $n - m$ of the constraints pass through the same vertex, so that the vertex is described by more than one basis. In the

tableau, degeneracy is visible as a zero in the left-hand column, or as a tie in the ratio test.



three constraints meeting at one vertex

If the ratio test is a tie, then θ^* may be zero: the pivot changes the basis but does *not* move the point. The cost then does not decrease, and it becomes possible in principle for the algorithm to return to a basis it has already visited, and loop forever. This is called **cycling**. It is rare in practice but it genuinely happens, and there are small explicit examples.

The fix is to break ties in a way that provably cannot cycle.

Definition 4.9 (Bland's Rule)

Bland's rule is to resolve both choices in the simplex method by smallest index:

- (i) among all j with $\bar{c}_j < 0$, choose the pivot column with the smallest index j ;
- (ii) among all rows attaining the minimum in the ratio test, choose the one whose *basic variable* $B(\ell)$ has the smallest index.

Theorem 4.10 (Bland)

The simplex method with Bland's rule terminates.

Proof sketch. Suppose the algorithm cycles. During a cycle the cost never changes, so every pivot has $\theta^* = 0$ and the point x is fixed throughout; let D be the set of variables that enter and leave the basis during the cycle, and let q be the largest index in D . Consider the iteration at which q leaves the basis, say with entering variable $p \in D$ (so $p < q$), and the (later) iteration at which q enters. One shows by comparing the reduced costs at these two iterations that the choice of p at the first iteration and the choice of q at the second are inconsistent with the smallest-index rule: at the second iteration the variable p would still have had negative reduced cost, and $p < q$, so p would have been chosen instead of q . This contradiction shows no cycle can occur, and since there are finitely many bases the algorithm terminates. $\square$

An alternative and more common fix in practice is *lexicographic* pivoting, which amounts to perturbing b to $b + (\varepsilon, \varepsilon^2, \dots, \varepsilon^m)$ for a formal infinitesimal ε , thereby making every basic solution non-degenerate.

Remark (How fast is the simplex method?). We have shown that the simplex method terminates, but the bound we get is terrible: there are $\binom{n}{m}$ bases and in principle we might visit a large fraction of them. In fact Klee and Minty constructed, for each n , a linear program on a slightly squashed n -dimensional cube for which Dantzig's original

pivoting rule visits all 2^n vertices. So the simplex method is not a polynomial-time algorithm, at least for that rule, and no pivoting rule is known to be polynomial.

Nevertheless the simplex method is superb in practice: on real problems it typically takes a number of pivots linear in m , and this behaviour can be made precise in various average-case and smoothed-analysis models. Meanwhile there *are* provably polynomial-time algorithms for linear programming – the ellipsoid method of Khachiyan and the interior point methods descended from Karmarkar’s algorithm, which are essentially the barrier methods of Section 1.4 applied to linear constraints. Modern solvers implement both families and choose between them.

§4.4 The Two-Phase Simplex Method

There is one loose end: the simplex method has to start at a basic feasible solution, and in the example above we were handed one by the slack variables. That is a piece of luck peculiar to problems of the form $Ax \leq b$ with $b \geq 0$. In general – for instance with $\geq$ constraints, where the surplus variables would need to be negative – finding an initial bfs is itself a non-trivial problem.

The trick is that finding a feasible point is itself a linear program that we can solve by the simplex method, and *that* problem always has an obvious starting point. We introduce **artificial variables**.

Given the standard-form problem ‘minimize $c \cdot x$ subject to $Ax = b$, $x \geq 0$ ’, where without loss of generality $b \geq 0$ (multiply rows by -1 if needed), consider the auxiliary problem

$$\begin{array}{ll} \text{minimize} & y_1 + \cdots + y_m \\ \text{subject to} & Ax + y = b, \\ & x, y \geq 0. \end{array}$$

This has the obvious basic feasible solution $x = 0$, $y = b$. Moreover its optimal value is 0 if and only if the original problem is feasible, and any solution with $y = 0$ gives a feasible x . So:

- **Phase I:** solve the auxiliary problem by the simplex method. If the optimal value is positive, the original problem is infeasible and we stop. Otherwise we finish with a basic feasible solution in which all the artificial variables are zero.
- **Phase II:** drop the artificial variables and the Phase I objective row, and run the simplex method on the original objective starting from the basis just found.

In practice we carry both objective rows through Phase I, so that the original objective row is correctly updated and ready for Phase II.

Example 4.11

Consider

$$\begin{array}{ll} \text{minimize} & 6x_1 + 3x_2 \\ \text{subject to} & x_1 + x_2 \geq 1, \\ & 2x_1 - x_2 \geq 1, \\ & 3x_2 \leq 2, \\ & x_1, x_2 \geq 0. \end{array}$$

Adding surplus variables z_1, z_2 and a slack z_3 ,

$$x_1 + x_2 - z_1 = 1, \quad 2x_1 - x_2 - z_2 = 1, \quad 3x_2 + z_3 = 2.$$

Setting $x = 0$ would force $z_1 = z_2 = -1$, which is infeasible, so we add artificial variables y_1, y_2 to the first two rows and minimize $y_1 + y_2$. The starting basis is $\{y_1, y_2, z_3\}$, giving the tableau (with the original objective in row 0 and the Phase I objective in row 0')

	x_1	x_2	z_1	z_2	z_3	y_1	y_2
0	6	3	0	0	0	0	0
0'	0	0	0	0	0	1	1
1	1	1	-1	0	0	1	0
1	2	-1	0	-1	0	0	1
2	0	3	0	0	1	0	0

This is not yet a valid tableau, because the reduced costs in the basic columns y_1, y_2 must be zero. Subtracting rows 1 and 2 from row 0' fixes this:

	x_1	x_2	z_1	z_2	z_3	y_1	y_2
0	6	3	0	0	0	0	0
-2	-3	0	1	1	0	0	0
1	1	1	-1	0	0	1	0
1	2	-1	0	-1	0	0	1
2	0	3	0	0	1	0	0

The Phase I cost is 2 (top-left of row 0' is -2). Pivoting on the boxed entry in column x_1 (ratios $1/1 = 1$ and $1/2 = \frac{1}{2}$, the latter smaller):

	x_1	x_2	z_1	z_2	z_3	y_1	y_2
-3	0	6	0	3	0	0	-3
$-\frac{1}{2}$	0	$-\frac{3}{2}$	1	$-\frac{1}{2}$	0	0	$\frac{3}{2}$
$\frac{1}{2}$	0	$\frac{3}{2}$	-1	$\frac{1}{2}$	0	1	$-\frac{1}{2}$
$\frac{1}{2}$	1	$-\frac{1}{2}$	0	$-\frac{1}{2}$	0	0	$\frac{1}{2}$
2	0	3	0	0	1	0	0

Now $\bar{c}_{x_2} = -\frac{3}{2} < 0$ in the Phase I row. The ratios are $\frac{1/2}{3/2} = \frac{1}{3}$ and $2/3$, so we pivot

on the boxed $\frac{3}{2}$:

	x_1	x_2	z_1	z_2	z_3	y_1	y_2
-5	0	0	4	1	0	-4	-1
0	0	0	0	0	0	1	1
$\frac{1}{3}$	0	1	$-\frac{2}{3}$	$\frac{1}{3}$	0	$\frac{2}{3}$	$-\frac{1}{3}$
$\frac{2}{3}$	1	0	$-\frac{1}{3}$	$-\frac{1}{3}$	0	$\frac{1}{3}$	$\frac{1}{3}$
1	0	0	2	-1	1	-2	1

The Phase I objective is now 0 and the artificial variables have left the basis, so Phase I is complete. Deleting the artificial columns and the Phase I row leaves

	x_1	x_2	z_1	z_2	z_3
-5	0	0	4	1	0
$\frac{1}{3}$	0	1	$-\frac{2}{3}$	$\frac{1}{3}$	0
$\frac{2}{3}$	1	0	$-\frac{1}{3}$	$-\frac{1}{3}$	0
1	0	0	2	-1	1

All reduced costs in row 0 are non-negative, so we are already optimal – Phase II has nothing to do. The optimal solution is $x_1 = \frac{2}{3}$, $x_2 = \frac{1}{3}$, $z_3 = 1$, with optimal cost $6 \cdot \frac{2}{3} + 3 \cdot \frac{1}{3} = 5$, which is consistent with the top-left entry -5.

Remark. It can happen that Phase I terminates with optimal value 0 but with an artificial variable still in the basis (at value zero). This is a degenerate situation; one can always pivot it out on any nonzero entry in a non-artificial column, or if the whole row is zero, delete the row as redundant.

§4.5 The Dual Simplex Method

The simplex method as described maintains primal feasibility and complementary slackness at every step, and works towards dual feasibility. There is a mirror-image algorithm, the **dual simplex method**, which maintains *dual* feasibility and complementary slackness and works towards primal feasibility. In tableau terms: we start with a tableau whose row 0 is non-negative but whose left-hand column has some negative entries, and we pivot to remove the negative entries while keeping row 0 non-negative.

Definition 4.12 (Dual Simplex Method)

Given a tableau with $\bar{c} \geq 0$:

- (1) If $B^{-1}b \geq 0$, stop: the solution is primal feasible, and hence optimal.
- (2) Otherwise choose a **pivot row** ℓ with $(B^{-1}b)_\ell < 0$.
- (3) If every entry of row ℓ (in the body) is non-negative, stop: the primal is infeasible.
- (4) Otherwise choose the **pivot column** j minimizing $|\bar{c}_j/a_{\ell j}|$ over all j with $a_{\ell j} < 0$.
- (5) Pivot on the entry $a_{\ell j}$ as usual, and go to (1).

The ratio test in step (4) is exactly what is needed to keep row 0 non-negative: after pivoting, $\bar{c}_k$ becomes $\bar{c}_k - (\bar{c}_j/a_{\ell j})a_{\ell k}$, and one checks that the minimizing choice keeps all of these non-negative. Notice that the roles of rows and columns, and of the two feasibility conditions, have swapped relative to the primal simplex method – which is precisely what one would expect from the fact that the dual of the dual is the primal.

The dual simplex method is extremely useful when we have already solved a problem and then change b (for instance by adding a new constraint). The old solution stays dual feasible but may become primal infeasible, so we can restart from it with the dual simplex method rather than solving from scratch.

Example 4.13

Consider

$$\begin{aligned} &\text{minimize} && 2x_1 + 3x_2 \\ &\text{subject to} && x_1 + x_2 \geq 2, \\ & && x_1 + 3x_2 \geq 3, \\ & && x_1, x_2 \geq 0. \end{aligned}$$

Subtracting surplus variables and then multiplying both constraint rows by -1 to make the slack variables basic, we get $-x_1 - x_2 + z_1 = -2$ and $-x_1 - 3x_2 + z_2 = -3$ with $z_1, z_2 \geq 0$, and the basis $\{z_1, z_2\}$ gives the tableau

	x_1	x_2	z_1	z_2
0	2	3	0	0
-2	-1	-1	1	0
-3	-1	-3	0	1

Row 0 is non-negative (dual feasible) but the left column is not (primal infeasible), so we use the dual simplex method. The most negative entry is -3 in row 2, so we take $\ell = 2$. The ratios $|\bar{c}_j/a_{2j}|$ for the negative entries of that row are $|2/(-1)| = 2$ and $|3/(-3)| = 1$, so we pivot on the boxed -3 in column x_2 :

	x_1	x_2	z_1	z_2
-3	1	0	0	1
-1	$-\frac{2}{3}$	0	1	$-\frac{1}{3}$
1	$\frac{1}{3}$	1	0	$-\frac{1}{3}$

Still one negative entry in the left column, so $\ell = 1$. The negative entries in that row are $-\frac{2}{3}$ in column x_1 and $-\frac{1}{3}$ in column z_2 , with ratios $|1/(-\frac{2}{3})| = \frac{3}{2}$ and $|1/(-\frac{1}{3})| = 3$; so we pivot on the boxed $-\frac{2}{3}$:

	x_1	x_2	z_1	z_2
$-\frac{9}{2}$	0	0	$\frac{3}{2}$	$\frac{1}{2}$
$\frac{3}{2}$	1	0	$-\frac{3}{2}$	$\frac{1}{2}$
$\frac{1}{2}$	0	1	$\frac{1}{2}$	$-\frac{1}{2}$

Now both the left column and row 0 are non-negative, so we have an optimal solution $x^* = (\frac{3}{2}, \frac{1}{2})$ with optimal cost $\frac{9}{2}$. Note that we never needed a Phase I.

§5 Network Problems

A great many of the linear programs that arise in practice have a very particular shape: the variables are indexed by the edges of a graph, and the constraints say that whatever flows into a vertex must flow out of it again. These are the **network flow** problems. They are important both because they model an enormous range of real situations – shipping goods, routing traffic, scheduling, matching – and because their special structure lets us solve them far more efficiently than by general-purpose simplex.

§5.1 Flows in Networks

Definition 5.1 (Directed Graph)

A **directed graph** (or **digraph**) is a pair $G = (V, E)$ where V is a finite set of **vertices** and $E \subseteq V \times V$ is a set of **edges**. We think of $(i, j) \in E$ as an arrow from i to j .

Definition 5.2 (Flow)

Let $G = (V, E)$ be a digraph on n vertices. A **flow** on G is an assignment of a real number x_{ij} to each edge $(i, j) \in E$, thought of as the amount of some commodity travelling along that edge. The flow is subject to:

- a vector $b \in \mathbb{R}^n$, where b_i is the amount injected into the network at vertex i from outside. If $b_i > 0$ we call i a **source** and if $b_i < 0$ a **sink**;
- a **cost** c_{ij} per unit of flow along (i, j) ;
- capacities $\underline{m}_{ij} \leq x_{ij} \leq \overline{m}_{ij}$ on each edge.

Definition 5.3 (Minimum Cost Flow)

The **minimum cost flow problem** is the linear program

$$\begin{aligned} & \text{minimize} && \sum_{(i,j) \in E} c_{ij} x_{ij} \\ & \text{subject to} && b_i + \sum_{j: (j,i) \in E} x_{ji} = \sum_{j: (i,j) \in E} x_{ij} \quad \text{for all } i \in V, \\ & && \underline{m}_{ij} \leq x_{ij} \leq \overline{m}_{ij} \quad \text{for all } (i,j) \in E. \end{aligned}$$

The first family of constraints is **conservation of flow**: what goes in must come out.

Summing the conservation equations over all i , every x_{ij} appears once on each side, so we get $\sum_i b_i = 0$. This is a necessary condition for feasibility, and it is obvious: the network has no storage, so the total injected must equal the total removed.

The conservation constraints can be written compactly.

Definition 5.4 (Incidence Matrix)

The **incidence matrix** of G is the matrix $A \in \mathbb{R}^{|V| \times |E|}$ whose column indexed by the edge (i, j) has $+1$ in row i , -1 in row j , and zeros elsewhere.

With this notation the conservation constraints are exactly $Ax = b$, and the minimum cost flow problem is a linear program in the (almost) standard form we have been studying. What makes network problems special is that A is a very sparse matrix with entries in $\{0, \pm 1\}$, and this has strong consequences – notably, if b and the capacities are integers then there is an integral optimal solution.

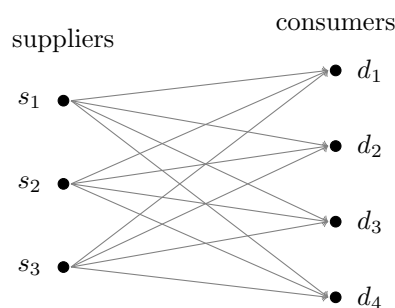
§5.2 The Transportation Problem

The most important special case is when the graph is bipartite.

Definition 5.5 (Transportation Problem)

Suppose we have n suppliers, where supplier i has s_i units available, and m consumers, where consumer j demands d_j units, with $\sum_i s_i = \sum_j d_j$. Shipping one unit from i to j costs c_{ij} . The **transportation problem** is

$$\begin{aligned} & \text{minimize} && \sum_{i=1}^n \sum_{j=1}^m c_{ij} x_{ij} \\ & \text{subject to} && \sum_{j=1}^m x_{ij} = s_i \quad \text{for } i = 1, \dots, n, \\ & && \sum_{i=1}^n x_{ij} = d_j \quad \text{for } j = 1, \dots, m, \\ & && x \geq 0. \end{aligned}$$



This looks like a special case, but it is in fact completely general.

Theorem 5.6

Every minimum cost flow problem with finite capacities, or with non-negative costs, is equivalent to a transportation problem.

Proof. First we normalize. We may assume $\underline{m}_{ij} = 0$ for all edges: substituting $x_{ij} = \underline{m}_{ij} + \tilde{x}_{ij}$ turns the constraint into $0 \leq \tilde{x}_{ij} \leq \bar{m}_{ij} - \underline{m}_{ij}$ and the conservation

equations into the same equations with

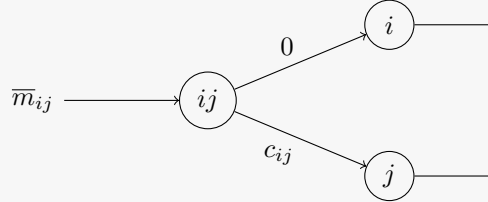
$$\tilde{b}_i = b_i + \sum_{j: (j,i) \in E} \underline{m}_{ji} - \sum_{j: (i,j) \in E} \underline{m}_{ij}.$$

(We are just secretly shipping the minimum amount along each edge before we start.) Next, if some capacity is infinite but all costs are non-negative, we may replace it by $\sum_i |b_i|$, since no optimal solution ever ships more than this along any edge (it would have to run round a cycle, and cycles cost nothing to remove when costs are non-negative). So assume all capacities are finite.

Now build a transportation problem. For each *vertex* i of the original graph, create a consumer with demand

$$\sum_{k: (i,k) \in E} \bar{m}_{ik} - b_i.$$

For each *edge* (i, j) of the original graph, create a supplier with supply $\bar{m}_{ij}$, joined to consumer i at cost 0 and to consumer j at cost c_{ij} .



Total supply is $\sum_{(i,j)} \bar{m}_{ij}$, and total demand is $\sum_i (\sum_k \bar{m}_{ik} - b_i) = \sum_{(i,j)} \bar{m}_{ij}$ since $\sum_i b_i = 0$; so the transportation problem is balanced.

Given a flow x in the original network, ship x_{ij} from supplier (i, j) to consumer j and the remaining $\bar{m}_{ij} - x_{ij}$ to consumer i . Then supplier (i, j) 's supply is exactly used up, and $0 \leq x_{ij} \leq \bar{m}_{ij}$ is exactly the requirement that both shipments are non-negative. The total received by consumer i is

$$\sum_{k: (i,k) \in E} (\bar{m}_{ik} - x_{ik}) + \sum_{k: (k,i) \in E} x_{ki},$$

and setting this equal to the demand $\sum_k \bar{m}_{ik} - b_i$ rearranges precisely to the conservation equation at i . Finally, the cost of the transportation solution is $\sum_{(i,j)} c_{ij} x_{ij}$, since the edges into consumer i are free. So the two problems are equivalent. $\square$

§5.3 Optimality for the Transportation Problem

The transportation problem is a linear program, so we know exactly what optimality means: primal feasibility, dual feasibility and complementary slackness. Let us compute what these say. Since there are two families of constraints, we take two families of multipliers: $\lambda \in \mathbb{R}^n$ for the suppliers and $\mu \in \mathbb{R}^m$ for the consumers. The Lagrangian is

$$\begin{aligned} L(x, \lambda, \mu) &= \sum_{i,j} c_{ij} x_{ij} - \sum_i \lambda_i \left(\sum_j x_{ij} - s_i \right) - \sum_j \mu_j \left(\sum_i x_{ij} - d_j \right) \\ &= \sum_{i,j} (c_{ij} - \lambda_i - \mu_j) x_{ij} + \sum_i \lambda_i s_i + \sum_j \mu_j d_j. \end{aligned}$$

Since $x \geq 0$, this has a finite infimum exactly when $c_{ij} - \lambda_i - \mu_j \geq 0$ for all i, j .

Theorem 5.7 (Optimality Conditions)

A feasible x is optimal for the transportation problem if there exist $\lambda \in \mathbb{R}^n$, $\mu \in \mathbb{R}^m$ with

- (i) $\lambda_i + \mu_j \leq c_{ij}$ for all (i, j) (dual feasibility)
- (ii) $(c_{ij} - \lambda_i - \mu_j) x_{ij} = 0$ for all (i, j) (complementary slackness)

Proof. Immediate from the general optimality conditions for a linear program, applied to the Lagrangian above. $\square$

Condition (ii) says: wherever we actually ship something, the dual constraint is tight. Note that the multipliers are not unique – adding a constant k to every λ_i and subtracting it from every μ_j changes nothing. We therefore normalize by setting $\lambda_1 = 0$, which leaves $m + n - 1$ unknowns.

To get started we need an initial basic feasible solution, and here there is a very simple recipe: the **north-west corner rule**. Start at the top-left cell of the table, ship as much as possible (the smaller of the remaining supply and remaining demand), cross off whichever of the row or column is exhausted, and repeat.

Theorem 5.8

The set of cells with $x_{ij} > 0$ in a basic feasible solution of the transportation problem forms a spanning tree of the bipartite graph, and hence contains exactly $m + n - 1$ cells.

We will not prove this, but it is easy to believe: there are $m + n$ constraints of which one is redundant (because $\sum s_i = \sum d_j$), so a basis has $m + n - 1$ elements; and a set of edges with no cycle and $m + n - 1$ elements on $m + n$ vertices is a spanning tree. The practical consequence is that the equations $\lambda_i + \mu_j = c_{ij}$ over the basic cells, together with $\lambda_1 = 0$, always have a unique solution – we can propagate along the tree.

§5.4 The Transportation Algorithm

We record the data in a **transportation tableau**, one cell per (i, j) , showing the current flow x_{ij} , the cost c_{ij} and the quantity $\lambda_i + \mu_j$, with the supplies down the right, the demands along the bottom, and λ, μ in the margins. The algorithm is then:

- (1) Find an initial basic feasible solution (e.g. by the north-west corner rule).
- (2) Solve $\lambda_i + \mu_j = c_{ij}$ over the basic cells, with $\lambda_1 = 0$.
- (3) If $\lambda_i + \mu_j \leq c_{ij}$ for every cell, stop – the solution is optimal.
- (4) Otherwise pick a cell (i, j) with $\lambda_i + \mu_j > c_{ij}$. Adding (i, j) to the spanning tree creates a unique cycle. Send θ units around this cycle (alternately $+\theta$ and $-\theta$), increasing θ until some basic cell is driven to zero. That cell leaves the basis and (i, j) enters.
- (5) Go to (2).

This is exactly the simplex method in disguise: step (3) is the check $\bar{c} \geq 0$, and step (4) is the ratio test.

Example 5.9

Take three suppliers with $s = (14, 10, 9)$ and four consumers with $d = (12, 5, 8, 8)$, and costs

$$C = \begin{pmatrix} 5 & 3 & 4 & 6 \\ 2 & 7 & 4 & 1 \\ 5 & 6 & 2 & 4 \end{pmatrix}.$$

The north-west corner rule gives

$$x = \begin{pmatrix} 12 & 2 & 0 & 0 \\ 0 & 3 & 7 & 0 \\ 0 & 0 & 1 & 8 \end{pmatrix},$$

with basic cells $(1, 1), (1, 2), (2, 2), (2, 3), (3, 3), (3, 4)$ – six of them, as promised. Complementary slackness gives

$$\lambda_1 + \mu_1 = 5, \quad \lambda_1 + \mu_2 = 3, \quad \lambda_2 + \mu_2 = 7, \quad \lambda_2 + \mu_3 = 4, \quad \lambda_3 + \mu_3 = 2, \quad \lambda_3 + \mu_4 = 4,$$

which with $\lambda_1 = 0$ solves to

$$\lambda = (0, 4, 2), \quad \mu = (5, 3, 0, 2).$$

It is convenient to record all of this in a single tableau, with the multipliers in the margins and each cell showing the current flow and the cost, written $x_{ij} \mid c_{ij}$:

	$\mu_1 = 5$	$\mu_2 = 3$	$\mu_3 = 0$	$\mu_4 = 2$	s_i
$\lambda_1 = 0$	12 5	2 3	0 4	0 6	14
$\lambda_2 = 4$	0 2	3 7	7 4	0 1	10
$\lambda_3 = 2$	0 5	0 6	1 2	8 4	9
d_j	12	5	8	8	

One checks at a glance that the rows sum to the supplies and the columns to the demands. Now we tabulate $\lambda_i + \mu_j$ against c_{ij} :

$\lambda_i + \mu_j$	$\mu_1 = 5$	$\mu_2 = 3$	$\mu_3 = 0$	$\mu_4 = 2$
$\lambda_1 = 0$	5 (5)	3 (3)	0 (4)	2 (6)
$\lambda_2 = 4$	9 (2)	7 (7)	4 (4)	6 (1)
$\lambda_3 = 2$	7 (5)	5 (6)	2 (2)	4 (4)

(costs c_{ij} in brackets). Cell $(2, 1)$ violates the condition badly: $9 > 2$. Cell $(2, 4)$ also violates it, as does $(3, 1)$; we pick $(2, 1)$.

Adding $(2, 1)$ to the tree creates the cycle $(2, 1) \rightarrow (1, 1) \rightarrow (1, 2) \rightarrow (2, 2) \rightarrow (2, 1)$. Putting θ into $(2, 1)$ forces $x_{11} \mapsto 12 - \theta$, $x_{12} \mapsto 2 + \theta$ and $x_{22} \mapsto 3 - \theta$. The binding constraint is $x_{22} \geq 0$, so $\theta = 3$ and cell $(2, 2)$ leaves. The new solution is

$$x = \begin{pmatrix} 9 & 5 & 0 & 0 \\ 3 & 0 & 7 & 0 \\ 0 & 0 & 1 & 8 \end{pmatrix},$$

with basic cells $(1, 1), (1, 2), (2, 1), (2, 3), (3, 3), (3, 4)$. Re-solving for the multipliers,

$$\lambda_1 + \mu_1 = 5, \lambda_1 + \mu_2 = 3, \lambda_2 + \mu_1 = 2, \lambda_2 + \mu_3 = 4, \lambda_3 + \mu_3 = 2, \lambda_3 + \mu_4 = 4,$$

giving $\lambda = (0, -3, -5)$ and $\mu = (5, 3, 7, 9)$. Checking again:

$\lambda_i + \mu_j$	$\mu_1 = 5$	$\mu_2 = 3$	$\mu_3 = 7$	$\mu_4 = 9$
$\lambda_1 = 0$	5 (5)	3 (3)	7 (4)	9 (6)
$\lambda_2 = -3$	2 (2)	0 (7)	4 (4)	6 (1)
$\lambda_3 = -5$	0 (5)	-2 (6)	2 (2)	4 (4)

Now $(2, 4)$ violates: $6 > 1$. (So does $(1, 3)$ and $(1, 4)$.) Taking $(2, 4)$, the cycle is $(2, 4) \rightarrow (2, 3) \rightarrow (3, 3) \rightarrow (3, 4) \rightarrow (2, 4)$, so $x_{23} \mapsto 7 - \theta$, $x_{33} \mapsto 1 + \theta$, $x_{34} \mapsto 8 - \theta$. The binding constraint is $x_{23} \geq 0$ at $\theta = 7$, giving

$$x = \begin{pmatrix} 9 & 5 & 0 & 0 \\ 3 & 0 & 0 & 7 \\ 0 & 0 & 8 & 1 \end{pmatrix}.$$

The new multipliers solve to $\lambda = (0, -3, 0)$ and $\mu = (5, 3, 2, 4)$, and now

$\lambda_i + \mu_j$	$\mu_1 = 5$	$\mu_2 = 3$	$\mu_3 = 2$	$\mu_4 = 4$
$\lambda_1 = 0$	5 (5)	3 (3)	2 (4)	4 (6)
$\lambda_2 = -3$	2 (2)	0 (7)	-1 (4)	1 (1)
$\lambda_3 = 0$	5 (5)	3 (6)	2 (2)	4 (4)

Every entry satisfies $\lambda_i + \mu_j \leq c_{ij}$, so we are optimal. The cost is

$$9 \cdot 5 + 5 \cdot 3 + 3 \cdot 2 + 7 \cdot 1 + 8 \cdot 2 + 1 \cdot 4 = 45 + 15 + 6 + 7 + 16 + 4 = 93.$$

Remark (Transshipment). A variant that often occurs is the **transshipment problem**, in which goods may pass through intermediate depots rather than going directly from supplier to consumer. Formally this is just a minimum cost flow problem on a graph with some vertices having $b_i = 0$, and by the reduction theorem above it can be converted into a transportation problem. In practice a slicker route is available: replace each cost c_{ij} by the cost $\hat{c}_{ij}$ of the *cheapest path* from i to j in the network (computable by any shortest-path algorithm), and then solve the resulting transportation problem with costs $\hat{c}$. Any optimal solution of that problem is realized by routing each shipment along its cheapest path.

§5.5 The Assignment Problem

A further special case, and one of the classics, is the following.

Definition 5.10 (Assignment Problem)

Given n workers and n jobs, and a cost c_{ij} of assigning worker i to job j , the **assignment problem** is to find a bijection σ from workers to jobs minimizing $\sum_i c_{i\sigma(i)}$.

Written as an optimization over $\{0, 1\}$ -matrices this is a combinatorial problem with

$n!$ candidates, which sounds hopeless. But it is exactly a transportation problem with $s_i = d_j = 1$ for all i, j :

$$\begin{aligned} & \text{minimize} && \sum_{i,j} c_{ij} x_{ij} \\ & \text{subject to} && \sum_j x_{ij} = 1 \text{ for each } i, \quad \sum_i x_{ij} = 1 \text{ for each } j, \quad x \geq 0. \end{aligned}$$

The feasible set of this linear program consists of the *doubly stochastic* matrices, and the corresponding combinatorial problem is recovered by the following classical fact, which we quote.

Theorem 5.11 (Birkhoff–von Neumann)

The extreme points of the set of doubly stochastic $n \times n$ matrices are exactly the permutation matrices.

Combining this with the fundamental theorem of linear programming, some optimal solution of the linear program is an extreme point, hence a permutation matrix, hence an honest assignment. So we may simply solve the transportation problem and read off the answer – an $n!$ -sized search has been replaced by a polynomial-time algorithm. This phenomenon, where the linear relaxation of an integer problem automatically has integral optima, is exactly what makes network problems so tractable, and it recurs in the maximum flow problem below.

§5.6 The Maximum Flow Problem

We now turn to a different, and in some ways more fundamental, network problem. There are no costs; instead every edge has a capacity, there is a single source (vertex 1) and a single sink (vertex n), and we want to push as much as possible from source to sink.

Definition 5.12 (Maximum Flow Problem)

Given a digraph $G = (V, E)$ on vertices $\{1, \dots, n\}$ with capacities $c_{ij} \geq 0$, the **maximum flow problem** is

$$\begin{aligned} & \text{maximize} && \delta \\ & \text{subject to} && \sum_{j: (i,j) \in E} x_{ij} - \sum_{j: (j,i) \in E} x_{ji} = \begin{cases} \delta & i = 1 \\ -\delta & i = n \\ 0 & \text{otherwise,} \end{cases} \\ & && 0 \leq x_{ij} \leq c_{ij} \quad \text{for all } (i, j) \in E. \end{aligned}$$

We call δ the **value** of the flow.

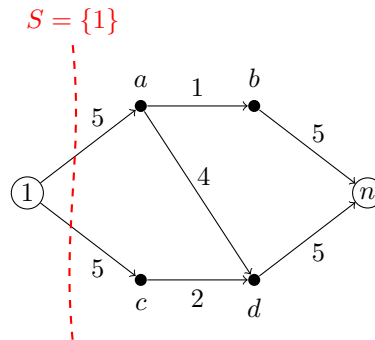
This is a minimum cost flow problem in disguise: add an edge from n back to 1 with cost -1 and infinite capacity, and set all $b_i = 0$; then minimizing the cost is the same as maximizing the flow round the added edge, which is δ . But there is a much better way to think about it.

Definition 5.13 (Cut)

A **cut** of G is a partition of V into two sets $(S, V \setminus S)$. Its **capacity** is

$$C(S) = \sum_{\substack{(i,j) \in E \\ i \in S, j \in V \setminus S}} c_{ij},$$

the total capacity of the edges crossing from S to $V \setminus S$.



The following inequality is the easy half of the story: no flow can beat any cut.

Proposition 5.14

Let x be a feasible flow with value δ , and let $(S, V \setminus S)$ be any cut with $1 \in S$ and $n \in V \setminus S$. Then $\delta \leq C(S)$.

Proof. For $X, Y \subseteq V$ write $f_x(X, Y) = \sum_{(i,j) \in E, i \in X, j \in Y} x_{ij}$ for the total flow from X to Y . Summing the conservation equations over $i \in S$, all terms vanish except the one at $i = 1$, so

$$\delta = \sum_{i \in S} \left(\sum_{j: (i,j) \in E} x_{ij} - \sum_{j: (j,i) \in E} x_{ji} \right) = f_x(S, V) - f_x(V, S).$$

Splitting V as $S \cup (V \setminus S)$ in both terms, the $f_x(S, S)$ contributions cancel, leaving

$$\delta = f_x(S, V \setminus S) - f_x(V \setminus S, S) \leq f_x(S, V \setminus S) \leq C(S),$$

using $f_x(V \setminus S, S) \geq 0$ and $x_{ij} \leq c_{ij}$. $\square$

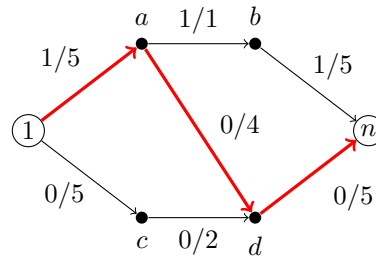
The remarkable fact is that this bound is always attained: some cut is exactly as good as the best flow. This is a theorem of Ford and Fulkerson, and it is a beautiful example of strong duality in a combinatorial setting – the maximum flow problem and the minimum cut problem are dual to each other.

Definition 5.15 (Augmenting Path)

Let x be a feasible flow. A path $v_0, v_1, \dots, v_k$ in the *undirected* sense – so consecutive vertices are joined by an edge in one direction or the other – is an **augmenting path** if

- $x_{v_i v_{i+1}} < c_{v_i v_{i+1}}$ for every edge traversed forwards, and
- $x_{v_{i+1} v_i} > 0$ for every edge traversed backwards.

The point is that along such a path we can increase the flow by some $\varepsilon > 0$ on the forward edges and decrease it by ε on the backward edges, and conservation still holds at every intermediate vertex. So if there is an augmenting path from 1 to n , the flow value can be increased and x was not optimal.



augmenting path 1, a, d, n

Theorem 5.16 (Max-Flow Min-Cut)

The maximum value of a feasible flow equals the minimum capacity of a cut separating 1 from n :

$$\delta^* = \min\{C(S) : S \subseteq V, 1 \in S, n \in V \setminus S\}.$$

Proof. By the proposition, $\delta^* \leq C(S)$ for every such cut, so it is enough to exhibit a cut with $C(S) = \delta^*$.

Let x be an optimal flow, and define

$$S = \{1\} \cup \{i \in V : \text{there is an augmenting path from 1 to } i\}.$$

Since x is optimal there is no augmenting path from 1 to n , so $n \notin S$ and $(S, V \setminus S)$ is a cut separating 1 from n . From the proof of the proposition,

$$\delta^* = f_x(S, V \setminus S) - f_x(V \setminus S, S),$$

and we claim both terms take their extreme values.

First, $f_x(V \setminus S, S) = 0$. Indeed, suppose $(v, u) \in E$ with $v \in V \setminus S$, $u \in S$ and $x_{vu} > 0$. Then we may extend an augmenting path from 1 to u by traversing (v, u) backwards, obtaining an augmenting path from 1 to v ; so $v \in S$, a contradiction.

Second, $f_x(S, V \setminus S) = C(S)$. Indeed, suppose $(u, v) \in E$ with $u \in S$, $v \in V \setminus S$ and $x_{uv} < c_{uv}$. Then we may extend an augmenting path from 1 to u by traversing (u, v) forwards, so $v \in S$, again a contradiction. Hence every edge from S to $V \setminus S$ is saturated.

Therefore $\delta^* = C(S) - 0 = C(S)$, as required. $\square$

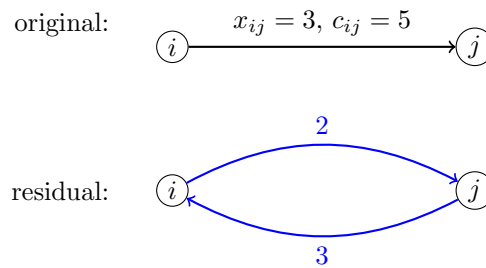
Notice that the proof is constructive: it tells us that if we cannot find an augmenting path, we are done, and it even hands us the certifying cut. This is the algorithm.

Definition 5.17 (Ford–Fulkerson Algorithm)

To find a maximum flow:

- (1) Start from any feasible flow, e.g. $x = 0$.
- (2) Search for an augmenting path from 1 to n . If there is none, stop – x is optimal.
- (3) Increase the flow along the path by as much as possible (the smallest of $c_{ij} - x_{ij}$ over forward edges and x_{ji} over backward edges), and go to (2).

The bookkeeping is easiest with the **residual network**: for each edge (i, j) we record a forward residual capacity $c_{ij} - x_{ij}$ and a backward residual capacity x_{ij} . An augmenting path is then just an ordinary directed path in the residual network, and the amount we can push is the smallest residual capacity along it.

**Example 5.18**

Consider the network drawn above, with capacities

$$c_{1a} = 5, \quad c_{ab} = 1, \quad c_{bn} = 5, \quad c_{1c} = 5, \quad c_{cd} = 2, \quad c_{dn} = 5, \quad c_{ad} = 4,$$

and start with $x = 0$.

- (i) The path $1, a, b, n$ is augmenting; the bottleneck is $c_{ab} = 1$, so we send 1 unit. Now $x_{1a} = x_{ab} = x_{bn} = 1$ and $\delta = 1$.
- (ii) The path $1, a, d, n$ is augmenting; the bottleneck is $c_{1a} - x_{1a} = 4$, so we send 4 units. Now $x_{1a} = 5, x_{ad} = 4, x_{dn} = 4$ and $\delta = 5$.
- (iii) The path $1, c, d, n$ is augmenting; the bottleneck is $c_{dn} - x_{dn} = 1$, so we send 1 unit. Now $x_{1c} = x_{cd} = 1, x_{dn} = 5$ and $\delta = 6$.

The final flow is $x_{1a} = 5, x_{ab} = 1, x_{bn} = 1, x_{ad} = 4, x_{1c} = 1, x_{cd} = 1, x_{dn} = 5$.

To see that this is optimal we compute the set S of vertices reachable from 1 by an augmenting path. The edge $1a$ is saturated, so a is not reachable that way; but $x_{1c} = 1 < 5$, so $c \in S$. From c , $x_{cd} = 1 < 2$, so $d \in S$. From d , the edge dn is saturated, but the edge ad carries flow $4 > 0$, so traversing it *backwards* puts $a \in S$. From a , the edge ab is saturated, so we get nothing new. Hence

$$S = \{1, a, c, d\},$$

and $n \notin S$, so this is a genuine cut. Its capacity is the total capacity of edges leaving S , namely ab and dn :

$$C(S) = c_{ab} + c_{dn} = 1 + 5 = 6 = \delta^*,$$

confirming optimality.

Remark. Finding the cut requires a little care: S consists of everything reachable by an augmenting path, and augmenting paths may traverse edges backwards, so one must chase the reachability to the end. When Ford–Fulkerson stops, the set of reachable vertices is automatically a minimum cut – that is exactly what the proof of the max-flow min-cut theorem showed.

Example 5.19

Here is a larger example in which a backward edge is genuinely needed. Take vertices $1, a, b, c, d, n$ with capacities

$$c_{1a} = 10, c_{1c} = 10, c_{ab} = 4, c_{ac} = 2, c_{ad} = 8, c_{bn} = 10, c_{cd} = 9, c_{db} = 6, c_{dn} = 10,$$

starting from $x = 0$.

- (i) Augment along $1, a, d, n$ by $\min(10, 8, 10) = 8$. Now $\delta = 8$, with $x_{1a} = 8$, $x_{ad} = 8$ (saturated) and $x_{dn} = 8$.
- (ii) Augment along $1, c, d, n$ by $\min(10, 9, 2) = 2$. Now $\delta = 10$, with $x_{1c} = 2$, $x_{cd} = 2$ and $x_{dn} = 10$ (saturated).
- (iii) Augment along $1, c, d, b, n$ by $\min(8, 7, 6, 10) = 6$. Now $\delta = 16$, with $x_{1c} = 8$, $x_{cd} = 8$, $x_{db} = 6$ (saturated) and $x_{bn} = 6$.
- (iv) Augment along $1, a, b, n$ by $\min(2, 4, 4) = 2$, the bottleneck being $c_{1a} - x_{1a} = 2$. Now $\delta = 18$, with $x_{1a} = 10$ (saturated), $x_{ab} = 2$ and $x_{bn} = 8$.

At this stage the only unsaturated edge out of 1 is $1c$, so $c \in S$; from c we have $x_{cd} = 8 < 9$, so $d \in S$; from d both dn and db are saturated, but $x_{ad} = 8 > 0$ so traversing ad backwards gives $a \in S$; from a we have $x_{ab} = 2 < 4$, so $b \in S$; and from b we have $x_{bn} = 8 < 10$, so $n \in S$. So we have found an augmenting path,

$$1 \rightarrow c \rightarrow d \xrightarrow{\text{backwards}} a \rightarrow b \rightarrow n,$$

with bottleneck $\min(2, 1, 8, 2, 2) = 1$, coming from the edge cd . Augmenting by 1 (and remembering to *decrease* x_{ad}) gives $\delta = 19$ and

$$x_{1a} = 10, x_{1c} = 9, x_{ab} = 3, x_{ac} = 0, x_{ad} = 7, x_{cd} = 9, x_{db} = 6, x_{bn} = 9, x_{dn} = 10.$$

Now $1a$ is saturated and $x_{1c} = 9 < 10$, so $c \in S$; but from c the edge cd is saturated and the only other edge at c is ac , which carries no flow. So $S = \{1, c\}$, and

$$C(S) = c_{1a} + c_{cd} = 10 + 9 = 19 = \delta^*.$$

So the flow of 19 is optimal, certified by the cut $\{1, c\}$. Notice that without the backward step in the fifth augmentation we would have stopped at $\delta = 18$: the flow of 8 units sent from a to d in step (i) was, with hindsight, a mistake, and the backward edge is precisely the mechanism by which the algorithm undoes it.

Theorem 5.20 (Termination)

If all capacities are integers, then the Ford–Fulkerson algorithm terminates in finitely many steps, and the maximum flow is integral.

Proof. If x is integral, then the bottleneck along any augmenting path is a positive integer, so the augmented flow is integral and the value increases by at least 1. Since δ is bounded above by the capacity of any cut, only finitely many augmentations can occur. Starting from $x = 0$, which is integral, the result follows. $\square$

The same argument works for rational capacities (clear denominators). For irrational capacities, the algorithm can in principle fail to terminate if the augmenting paths are chosen badly – choosing shortest augmenting paths always works, and gives the Edmonds–Karp algorithm.

§5.7 An Application: Bipartite Matching

The integrality in the theorem above is what makes max-flow such a powerful combinatorial tool: it turns questions about $\{0, 1\}$ -valued combinatorial objects into questions about flows.

Theorem 5.21

Every k -regular bipartite graph has a perfect matching.

Proof. Let G be a bipartite graph with parts L and R each of size N , with every vertex having degree k . (The parts have equal size because counting edges gives $k|L| = |E| = k|R|$.) Build a flow network: add a source 1 joined to every vertex of L with capacity 1, add a sink n joined from every vertex of R with capacity 1, and direct every edge of G from L to R with capacity 1.

The cut $S = \{1\}$ has capacity N , so $\delta^* \leq N$. On the other hand, sending $1/k$ along every edge of G and 1 along every edge from the source and to the sink gives a feasible flow of value N (each vertex of L receives 1 and sends out $k \cdot \frac{1}{k} = 1$). So $\delta^* = N$.

By the previous theorem, since all capacities are integers, there is an *integral* flow of value N . Every edge then carries flow 0 or 1, and every vertex of L has exactly one outgoing edge with flow 1 (it receives exactly 1 from the source), and similarly every vertex of R has exactly one incoming edge with flow 1. The edges carrying flow 1 therefore form a perfect matching. $\square$

Remark. The same technique proves Hall’s marriage theorem: a bipartite graph with parts L, R has a matching saturating L if and only if $|N(A)| \geq |A|$ for every $A \subseteq L$. One shows that a cut of capacity less than $|L|$ in the network above corresponds exactly to a violating set A , and then applies max-flow min-cut.

§6 Game Theory

We close with a topic that at first sight has nothing to do with optimization: what should you do when the thing you are optimizing against is another person, who is optimizing against you? The answer, remarkably, is that a two-player zero-sum game is exactly a pair of dual linear programs, and the fundamental theorem of such games – the minimax theorem – is exactly strong duality. This is one of the most satisfying appearances of the theory we have built.

§6.1 Two-Player Zero-Sum Games

Definition 6.1 (Zero-Sum Game)

A **two-person zero-sum game** consists of two players, P_1 and P_2 , with m and n available actions respectively, and a **payoff matrix** $A \in \mathbb{R}^{m \times n}$. If P_1 plays i and P_2 plays j , then P_1 wins the amount a_{ij} and P_2 loses the same amount.

So P_1 chooses a row and wants the entry to be large; P_2 chooses a column and wants it to be small. ‘Zero-sum’ means that one player’s gain is exactly the other’s loss – there is nothing to cooperate over.

Suppose first that the players move in sequence, and P_1 must move first. If P_1 plays row i then P_2 , seeing this, will respond with the column minimizing a_{ij} . So P_1 should solve

$$\max_i \min_j a_{ij}.$$

Symmetrically, if P_2 moves first then P_2 should solve

$$\min_j \max_i a_{ij}.$$

It is always the case that

$$\max_i \min_j a_{ij} \leq \min_j \max_i a_{ij},$$

since $\min_{j'} a_{ij'} \leq a_{ij} \leq \max_{i'} a_{i'j}$ for every i, j ; taking the max over i on the left and the min over j on the right preserves this. Moving second is (weakly) an advantage: you get to see what your opponent did.

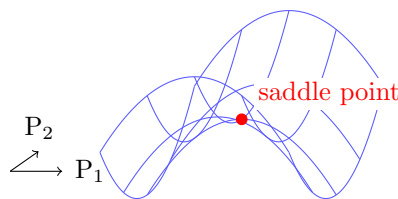
Definition 6.2 (Saddle Point)

An entry $a_{i^*j^*}$ is a **saddle point** if it is simultaneously the minimum of its row and the maximum of its column, that is

$$a_{i^*j} \geq a_{i^*j^*} \geq a_{ij^*} \quad \text{for all } i, j.$$

Equivalently, $\max_i \min_j a_{ij} = a_{i^*j^*} = \min_j \max_i a_{ij}$. In this case $a_{i^*j^*}$ is called the **value** of the game.

The name comes from the geometry: at a saddle point the payoff surface curves up in one direction and down in the other, like a mountain pass.



Example 6.3

Take

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}.$$

Row minima are 1 and 3, so $\max_i \min_j a_{ij} = 3$. Column maxima are 3 and 4, so $\min_j \max_i a_{ij} = 3$. These agree, so $(2, 1)$ is a saddle point and the value of the game is 3. Both players should play their part of the saddle point regardless of who moves first, and neither can do better by deviating.

Example 6.4

Take

$$A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}.$$

Row minima are 2 and 1, so $\max_i \min_j a_{ij} = 2$. Column maxima are 4 and 3, so $\min_j \max_i a_{ij} = 3$. These do not agree, and there is no saddle point.

In this game there is no stable pair of choices. If P_1 plays row 1 then P_2 prefers column 2; but against column 2, P_1 prefers row 2; against row 2, P_2 prefers column 1; against column 1, P_1 prefers row 1 – and we are back where we started. Whatever either player does, if the other knows it they can exploit it.

§6.2 Mixed Strategies

The resolution of the second example is one of the great ideas of game theory: the players should randomize. If P_1 picks a row at random according to a probability distribution, then P_2 cannot know which row will be played, only the distribution, and so cannot exploit the choice exactly.

Definition 6.5 (Mixed Strategy)

A **mixed strategy** for P_1 is a vector $p \in \mathbb{R}^m$ with $p \geq 0$ and $\sum_i p_i = 1$, where p_i is the probability of playing action i . Similarly a mixed strategy for P_2 is $q \in \mathbb{R}^n$ with $q \geq 0$, $\sum_j q_j = 1$. A strategy which plays one action with probability 1 is called **pure**.

If P_1 plays p and P_2 plays q , the expected payoff to P_1 is

$$\sum_{i,j} p_i a_{ij} q_j = p \cdot (Aq) = p^T Aq.$$

If P_1 plays p and P_2 responds with a pure strategy j , the expected payoff is $\sum_i a_{ij} p_i = (A^T p)_j$; and since a mixed strategy is an average of pure ones, P_2 can do no better than the best pure response. So P_1 wants to solve

$$\begin{aligned} & \text{maximize} && \min_j (A^T p)_j \\ & \text{subject to} && \sum_i p_i = 1, \quad p \geq 0. \end{aligned}$$

This is not yet a linear program, but the standard trick of introducing a variable for the minimum makes it one. Writing $e = (1, 1, \dots, 1)^T$ (of whatever length is needed),

$$(P_1) \quad \text{maximize} \quad v$$

$$\begin{aligned} \text{subject to } & A^T p \geq ve, \\ & e \cdot p = 1, \quad p \geq 0. \end{aligned}$$

Similarly, P_2 wants to minimize the maximum expected loss, giving

$$\begin{aligned} (P_2) \quad & \text{minimize } w \\ \text{subject to } & Aq \leq we, \\ & e \cdot q = 1, \quad q \geq 0. \end{aligned}$$

Both are honest linear programs, in the variables (p, v) and (q, w) respectively. And now the punchline.

Theorem 6.6

The problems (P_1) and (P_2) are dual to each other.

Proof. Introduce a slack variable $s \geq 0$ to write P_2 's constraints as $Aq + s = we$, and take multipliers $\lambda_1 \in \mathbb{R}^m$ for these and $\lambda_2 \in \mathbb{R}$ for the normalization $e \cdot q = 1$. The Lagrangian is

$$\begin{aligned} L(w, q, s, \lambda) &= w + \lambda_1 \cdot (Aq + s - we) - \lambda_2(e \cdot q - 1) \\ &= w(1 - \lambda_1 \cdot e) + (A^T \lambda_1 - \lambda_2 e) \cdot q + \lambda_1 \cdot s + \lambda_2. \end{aligned}$$

Now: w is unrestricted, so we need $\lambda_1 \cdot e = 1$; $q \geq 0$, so we need $A^T \lambda_1 - \lambda_2 e \geq 0$; and $s \geq 0$, so we need $\lambda_1 \geq 0$. Given all of these the infimum is λ_2 . Hence the dual problem is

$$\begin{aligned} \text{maximize } & \lambda_2 \\ \text{subject to } & A^T \lambda_1 \geq \lambda_2 e, \\ & e \cdot \lambda_1 = 1, \quad \lambda_1 \geq 0, \end{aligned}$$

which is exactly (P_1) under the identification $\lambda_1 = p$, $\lambda_2 = v$. $\square$

Theorem 6.7 (Minimax Theorem)

For any payoff matrix A ,

$$\max_p \min_q p^T A q = \min_q \max_p p^T A q,$$

where the maximum and minimum are over mixed strategies. The common value v is called the **value** of the game.

Proof. The left-hand side is the optimal value of (P_1) and the right-hand side is the optimal value of (P_2) , and we have just shown these are dual linear programs. Both are feasible – take p and q uniform, and v, w suitably extreme – and both are bounded, since $\min_{i,j} a_{ij} \leq p^T A q \leq \max_{i,j} a_{ij}$ always. So strong duality applies and the optimal values agree. $\square$

This is a genuinely surprising theorem. It says that in a zero-sum game, once we allow randomization, there is no advantage to moving second: P_1 can guarantee an expected

payoff of at least v no matter what P_2 does, and P_2 can guarantee losing at most v no matter what P_1 does. The game has a well-defined value, and there is no secret to keep.

The complementary slackness conditions for this dual pair are worth recording, since they are what we use to compute.

Theorem 6.8 (Optimality Conditions for Games)

Mixed strategies p, q are optimal for P_1 and P_2 respectively, with value v , if and only if

$$(i) \quad A^T p \geq ve, \quad e \cdot p = 1, \quad p \geq 0; \quad (\text{primal feasibility})$$

$$(ii) \quad Aq \leq ve, \quad e \cdot q = 1, \quad q \geq 0; \quad (\text{dual feasibility})$$

$$(iii) \quad v = p^T Aq. \quad (\text{complementary slackness})$$

Proof. Complementary slackness for the dual pair above reads $p \cdot (Aq - we) = 0$ and $q \cdot (A^T p - ve) = 0$. Since $e \cdot p = e \cdot q = 1$, these say $p^T Aq = w$ and $p^T Aq = v$ respectively, so at an optimum $v = w = p^T Aq$. $\square$

The practical content of (iii) is the following pair of statements, which we will use in a moment:

- if $p_i > 0$ then the i th constraint of P_2 's problem is tight, i.e. $(Aq)_i = v$;
- if $q_j > 0$ then the j th constraint of P_1 's problem is tight, i.e. $(A^T p)_j = v$.

In words: every action that a player actually uses with positive probability must give exactly the value of the game against the opponent's optimal strategy. If some action gave strictly less, the player would drop it.

§6.3 Finding Optimal Strategies

In principle we can always solve the linear program by the simplex method, but for small games there are shortcuts.

- (1) **Look for a saddle point.** If the payoff matrix has one, the corresponding pure strategies are optimal for both players and we are done.
- (2) **Eliminate dominated actions.** If $a_{ij} \geq a_{i'j}$ for every j , then row i **dominates** row i' : P_1 can never do worse by playing i instead of i' , so there is an optimal strategy with $p_{i'} = 0$, and we may delete row i' . Similarly, if $a_{ij} \leq a_{ij'}$ for every i then column j dominates column j' (from P_2 's point of view, smaller is better), and we may delete column j' .
- (3) **Reduce to a small linear program.** A $2 \times n$ or $m \times 2$ game reduces to a one-dimensional problem that can be solved graphically.
- (4) **Use complementary slackness** to recover the other player's strategy once you have one of them.

Example 6.9

Let us return to the game with no saddle point,

$$A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix},$$

and solve it properly. Write P_1 's mixed strategy as $p = (p, 1 - p)$. Against P_2 's two pure strategies the expected payoffs are

$$(A^T p)_1 = 4p + (1 - p) = 1 + 3p, \quad (A^T p)_2 = 2p + 3(1 - p) = 3 - p,$$

and P_1 wants to maximize the smaller of these. The first is increasing in p and the second decreasing, so the maximum of the minimum is where they cross:

$$1 + 3p = 3 - p \implies p = \frac{1}{2}, \quad v = \frac{5}{2}.$$

Similarly, writing $q = (q, 1 - q)$ for P_2 ,

$$(Aq)_1 = 4q + 2(1 - q) = 2 + 2q, \quad (Aq)_2 = q + 3(1 - q) = 3 - 2q,$$

and P_2 minimizes the larger of these, which happens where $2 + 2q = 3 - 2q$, that is $q = \frac{1}{4}$, again with value $\frac{5}{2}$.

So the value of the game is $\frac{5}{2}$, with $p^* = (\frac{1}{2}, \frac{1}{2})$ and $q^* = (\frac{1}{4}, \frac{3}{4})$. Notice that $\frac{5}{2}$ lies strictly between $\max_i \min_j a_{ij} = 2$ and $\min_j \max_i a_{ij} = 3$: randomizing has genuinely bought P_1 something over any pure strategy, and it has cost P_2 less than any pure strategy would have.

Notice also the structure of the answer: at the optimum, *both* of P_2 's pure strategies give P_1 the same expected payoff. This is complementary slackness at work – since $q_1, q_2 > 0$, both of P_1 's constraints must be tight. Equivalently, P_1 chooses p precisely so as to make P_2 indifferent between their options; if P_2 were not indifferent, they would have a strictly better reply and could exploit P_1 .

Example 6.10

Consider the game with payoff matrix

$$A = \begin{pmatrix} 2 & 3 & 4 \\ 3 & 1 & \frac{1}{2} \\ 1 & 3 & 2 \end{pmatrix}.$$

First, is there a saddle point? The row minima are $2, \frac{1}{2}, 1$, so $\max_i \min_j = 2$; the column maxima are $3, 3, 4$, so $\min_j \max_i = 3$. These differ, so there is no saddle point and we will need mixed strategies.

Next, look for domination. Row 1 is $(2, 3, 4)$ and row 3 is $(1, 3, 2)$; since $2 \geq 1$, $3 \geq 3$ and $4 \geq 2$, row 1 dominates row 3, so we may take $p_3 = 0$ and work with

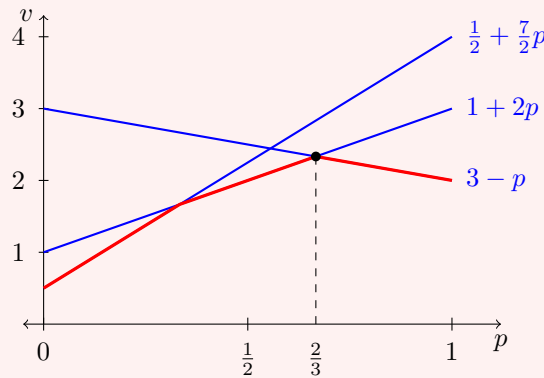
$$\tilde{A} = \begin{pmatrix} 2 & 3 & 4 \\ 3 & 1 & \frac{1}{2} \end{pmatrix}.$$

Write P_1 's strategy as $p = (p, 1 - p, 0)$. Then P_1 's problem becomes

$$\text{maximize } v$$

$$\begin{aligned} \text{subject to } & 2p + 3(1 - p) \geq v, \\ & 3p + 1(1 - p) \geq v, \\ & 4p + \frac{1}{2}(1 - p) \geq v, \\ & 0 \leq p \leq 1, \end{aligned}$$

that is, $v \leq \min \{3 - p, 1 + 2p, \frac{1}{2} + \frac{7}{2}p\}$, and we want to choose p to make this minimum as large as possible. Since this is a one-dimensional problem we can just draw the three lines and look at their lower envelope.



The lower envelope, drawn in red, is maximized where the lines $3 - p$ and $1 + 2p$ cross, at $p = \frac{2}{3}$, giving $v = \frac{7}{3}$. (The third line, $\frac{1}{2} + \frac{7}{2}p$, takes the value $\frac{1}{2} + \frac{7}{3} = \frac{17}{6} > \frac{7}{3}$ there, so it is not binding.) So

$$p^* = \left(\frac{2}{3}, \frac{1}{3}, 0\right), \quad v = \frac{7}{3}.$$

Now we use complementary slackness to find P_2 's strategy q . Since $p_1, p_2 > 0$, the first two constraints of P_2 's problem must be tight:

$$(Aq)_1 = 2q_1 + 3q_2 + 4q_3 = \frac{7}{3}, \quad (Aq)_2 = 3q_1 + q_2 + \frac{1}{2}q_3 = \frac{7}{3}.$$

Also, the third constraint of P_1 's problem is slack (we computed $\frac{17}{6} > \frac{7}{3}$), so complementary slackness forces $q_3 = 0$. With $q_1 + q_2 = 1$, the first equation gives

$$2q_1 + 3(1 - q_1) = \frac{7}{3} \implies 3 - q_1 = \frac{7}{3} \implies q_1 = \frac{2}{3},$$

and one checks that the second equation is satisfied: $3 \cdot \frac{2}{3} + \frac{1}{3} = \frac{7}{3}$. So

$$q^* = \left(\frac{2}{3}, \frac{1}{3}, 0\right).$$

Finally we should sanity check the answer against the optimality conditions. We have $A^T p^* = \left(\frac{7}{3}, \frac{7}{3}, \frac{17}{6}\right) \geq \frac{7}{3}e$ and $Aq^* = \left(\frac{7}{3}, \frac{7}{3}, \frac{5}{3}\right) \leq \frac{7}{3}e$, and $p^{*T} Aq^* = \frac{7}{3}$. All three conditions hold, so p^* and q^* really are optimal and the value of the game is $\frac{7}{3}$.

Remark. It is worth noticing what this answer means. P_1 's value from playing the pure strategy 'always row 1' is only 2 (their opponent would always answer with column 1), and from 'always row 2' is only $\frac{1}{2}$. By mixing the two in the ratio 2 : 1, P_1 guarantees $\frac{7}{3} > 2$. Randomization is strictly better than any deterministic play. That this is possible at all, and that the resulting value is exactly matched by the opponent's best defence, is the content of the minimax theorem – which is to say, of linear programming duality.